United States Patent	12400104
Kind Code	B1
Date of Patent	August 26, 2025
Inventor(s)	Passban; Peyman et al.

Updating machine learning models

Abstract

A system may share updates to a machine learning model between a first device and a second device. The first device may update parameters of a model operating on the first device in response to processing input data. The first device may generate model update data based on the updated parameters, and send the model update data to the second device. The first and second devices may agree on one or more protocols for generating the model update data to obscure details of the model, and/or reduce a communication cost of sending the model update data (e.g., relative to a size of the model). The second device may aggregate model update data (e.g., from multiple devices), determine second model update data, and send the second model update data to the first device. The first device may update its own model based on the second model update data.

Inventors: **Passban; Peyman (Vaughan, CA), Chung; Clement (Toronto, CA), Roostaeyan; Gazelle Tanya (Saratoga, CA), Gupta; Rahul (Waltham, MA), Dupuy; Christophe (Cambridge, MA)**

Applicant: **Amazon Technologies, Inc.** (Seattle, WA)

Family ID: **1000005931848**

Assignee: **Amazon Technologies, Inc. (Seattle, WA)**

Appl. No.: **17/490287**

Filed: **September 30, 2021**

Publication Classification

Int. Cl.: **G06N3/045** (20230101); **G06F18/21** (20230101); **G06F18/214** (20230101)

U.S. Cl.:

CPC **G06N3/045** (20230101); **G06F18/214** (20230101); **G06F18/217** (20230101);

Field of Classification Search

CPC: G06N (3/045); G06N (3/08); G06F (18/21); G06F (18/214); G06F (18/217)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
11237880	12/2021	Raumann	N/A	G06F 9/544
12136034	12/2023	Dimitriadis	N/A	G06N 3/08
12271810	12/2024	Reddi	N/A	G06N 20/20
2018/0060759	12/2017	Chu	N/A	G06N 20/00
2018/0082181	12/2017	Brothers	N/A	G06N 3/045
2018/0293517	12/2017	Browne	N/A	G06F 8/437
2019/0042878	12/2018	Sheller	N/A	G06F 21/6245
2019/0258924	12/2018	Hamidouche	N/A	G06N 3/08
2020/0104705	12/2019	Bhowmick	N/A	G06N 3/044
2020/0104706	12/2019	Sandler	N/A	G06N 3/045
2020/0242514	12/2019	McMahan	N/A	G06N 7/01
2021/0065038	12/2020	Gu	N/A	G06F 7/24
2021/0110211	12/2020	Grindstaff	N/A	G06Q 10/04
2021/0168195	12/2020	O	N/A	H04L 67/10
2021/0182661	12/2020	Li	N/A	H04L 41/16
2022/0398500	12/2021	Singhal	N/A	G06N 3/098
2023/0169350	12/2022	Louizos	706/25	G06N 3/045

Primary Examiner: Baldwin; Randall K.

Attorney, Agent or Firm: Pierce Atwood LLP

Background/Summary

BACKGROUND

(1) Speech recognition systems have progressed to the point where humans can interact with computing devices using their voices. Such systems employ techniques to identify the words spoken by a human user based on the various qualities of a received audio input. Speech recognition combined with natural language understanding processing techniques enable speech-based user control of a computing device to perform tasks based on the user's spoken commands. Speech recognition and natural language understanding processing techniques may be referred to collectively or separately herein as speech processing. Speech processing may also involve converting a user's speech into text data which may then be provided to various text-based software applications.

(2) Speech processing may be used by computers, hand-held devices, telephone computer systems, kiosks, and a wide variety of other devices to improve human-computer interactions.

Description

BRIEF DESCRIPTION OF DRAWINGS

(1) For a more complete understanding of the present disclosure, reference is now made to the following description taken in conjunction with the accompanying drawings.

(2) FIG. **1** is a conceptual diagram illustrating example operations of a system performing secure updates of machine learning models across devices, according to embodiments of the present disclosure.

(3) FIGS. **2**A-**2**D are conceptual diagrams illustrating various examples of sending and receiving secure updates for machine learning models, according to embodiments of the present disclosure.

(4) FIG. **3**A is a flowchart describing example operations of a method of generating, sending, and receiving secure updates for machine learning models, according to embodiments of the present disclosure.

(5) FIG. **3**B is a flowchart describing example operations of a method of receiving, aggregating, and sending secure updates for machine learning models, according to embodiments of the present disclosure.

(6) FIG. **4** is a conceptual diagram illustrating example operations of a device training and updating a machine learning model, according to embodiments of the present disclosure.

(7) FIG. **5**A is a conceptual diagram illustrating an example of aggregating model updates from multiple devices, according to embodiments of the present disclosure.

(8) FIG. **5**B is a conceptual diagram illustrating a first example of using model update from a first device to update a model on a second device, according to embodiments of the present disclosure.

(9) FIG. **5**C is a conceptual diagram illustrating a second example of using model update from a first device to update a model on a second device, according to embodiments of the present disclosure.

(10) FIG. **5**D is a conceptual diagram illustrating an example of a second device/system receiving a model update from a first device, and aggregating the model update data into a second model, according to embodiments of the present disclosure.

(11) FIG. **5**E is a conceptual diagram illustrating an example of generating model update data based on an updated second model, and sending model update data to the first device, according to embodiments of the present disclosure.

(12) FIG. **6** is a conceptual diagram of components of the system, according to embodiments of the present disclosure.

(13) FIG. **7** is a conceptual diagram of an ASR component, according to embodiments of the present disclosure.

(14) FIG. **8** is a conceptual diagram of how natural language processing is performed, according to embodiments of the present disclosure.

(15) FIG. **9** is a conceptual diagram of text-to-speech components according to embodiments of the present disclosure.

(16) FIG. **10** is a conceptual diagram of an image processing component according to embodiments of the present disclosure.

(17) FIG. **11** is a block diagram conceptually illustrating example components of a device, according to embodiments of the present disclosure.

(18) FIG. **12** is a block diagram conceptually illustrating example components of a system, according to embodiments of the present disclosure.

(19) FIG. **13** illustrates an example of a computer network for use with the overall system, according to embodiments of the present disclosure.

DETAILED DESCRIPTION

(20) A computer system may use one or more machine learning models to process input data to make inferences and/or predictions. Such machine learning models may include artificial neural networks (NN) such as convolutional networks, recurrent neural networks (RNN), long short-term

memory (LSTM), transformers, conformers, etc. An NN may be made up of one or more layers, with a layer including one or more cells (also referred to as artificial neurons). A cell may include a number of inputs and outputs. A cell may receive inputs and generate output(s) by performing one or more mathematical and/or logical operations described by one or more parameters of the cell. For example, a cell may take a weighted sum of input values and apply an activation function on the result to yield the output(s). Some cells may additionally perform operations based on a previous output, a memory state, a context signal, etc. A computer system may train an NN by various techniques to improve results of the NN with respect to a dataset by adjusting some or all of the parameters.

(21) From time to time, NN model parameters may be updated based on a new dataset or some other improvement. In some cases, a first device operating the model may share or otherwise contribute to the generation of model update data for a second device operating the same or otherwise similar model. However, some updates may be based on user data or other confidential information, which the system may prevent from leaving the first device for improving the second device. To keep the data confidential or otherwise secret, the first device may share information limited to model parameters derived from the data and from which the data cannot be reconstructed. Because NN models can get large, transferring all model parameters between the devices may consume significant bandwidth.

(22) Offered are a system, method, and other technology for improving the security of updating machine learning models over a network while reducing the communication cost of sending updates. Devices may agree on protocols for sending and receiving model update data. The protocols may describe operations for determining a portion of an updated model to send and/or how to transform the portion to obscure the model from third-party observers. For example, a protocol may specify a subset of layers of a neural network model to be transferred, and further specify one or more operations for transforming the parameters of the subset. In various implementations, a protocol may specify a random subset of model parameters, a reordering of model layers, and/or operations for projecting parameters by changing the size and/or dimensions of the layers prior to transmission. In some implementations, a protocol may specify a subset of layers and/or parameters representing a fraction of the size of the overall model, thereby reducing bandwidth used for transferring update data. In some implementations, the devices may alter the protocol to, for example, change which parameters are updated at each iteration and/or to further obscure model data. These and other features of the disclosure are provided as examples, and maybe used in combination with each other and/or with additional features described herein.

(23) The system may be configured to incorporate user permissions and may only perform activities disclosed herein if approved by a user. As such, the systems, devices, components, and techniques described herein would be typically configured to restrict processing where appropriate and only process user information in a manner that ensures compliance with all appropriate laws, regulations, standards, and the like. The system and techniques can be implemented on a geographic basis to ensure compliance with laws in various jurisdictions and entities in which the components of the system and/or user are located.

(24) FIG. **1** is a conceptual diagram illustrating example operations of a system **100** performing secure updates of machine learning models across devices, according to embodiments of the present disclosure. The system **100** may employ one or more machine learning models to perform various functions including, but not limited to, audio processing such as acoustic event detection, ASR, speaker recognition, NLU, TTS, etc.; image processing such as text recognition, object detection, facial recognition, etc.; and/or other types of processing based on, for example, sensor data, radar/LIDAR/sonar data, financial data, medical data, etc. An example system **100** performing various processing operations using machine learning models is shown in FIG. **6** and described below.

(25) The system **100** shown in FIG. **1** includes a first device **110** and a second device/system **120**

communicating via one or more computer networks such as the network **199**. The system **100** may include additional devices **110** and second device/systems **120**, examples of which are illustrated in and described with reference to FIGS. **11** through **13**. The first device **110** may store and operate one or more machine learning models, such as the first model **115**. The first model **115** may be, for example and without limitation, an ASR model such as described below with reference to FIG. **7**, an NLU model such as described below with reference to FIG. **8**, a TTS model such as described below with reference to FIG. **9**, an image processing model such as described below with reference to FIG. **10**, etc. A user **5** may operate the first device **110** to perform operations using one or more machine learning models **115** to, for example, process spoken commands or process image data received by a camera of the first device **110**. During operation, the first device **110** may receive input data (e.g., from the user via a microphone, camera, and/or touchscreen) and process it using the first model **115** to determine output data. The first device **110** may train **140** the first model **115** based on results of the processing to improve performance of the first model **115**. Training of the first machine learning model **115** by the first device **110** is illustrated in FIG. **4** and described in further detail below.

(26) Occasionally, the first device **110** may send model update data—e.g., representing parameters of the first model **115**—to the second device/system **120**. The first device **110** may generate **145** the model update data based on one or more protocols **150** shared among the first device **110** and the second device/system **120**. The protocol **150** may, for example, specify a portion of the first model **115** to use to generate the model update data and one or more operations for transforming the portion of the first model **115** to generate first model update data **155**. For example, the protocol **150** may specify a certain subset of parameters and/or layers of the first model **115** to send to the second device/system **120**. The protocol **150** may further specify how the subset of parameters and/or layers should be selected, reordered, projected, etc., to generate the first model update data **155** and send it to the second device/system **120**. In some implementations, the first device **110** may send an indication of the protocol **150** used to generate the first model update data **155** to the second device/system **120**. The receiving device **110**/system **120** may use the indication to retrieve the protocol from a storage or memory, and use the protocol to decode the model update data (e.g., to determine the model parameters represented in the model update data). Various examples of parameter selection and transformation are described below with reference to FIGS. **2**A through **2**D. The first device **110** may send the resulting first model update data **155** to the second device/system **120**.

(27) The second device/system **120** may receive the first model update data **155** and, using the protocol **150**, process it to update a second model **125** maintained by the second device/system **120**. The second device/system **120** may aggregate **160** the first model update data **155** as well as additional model update data **165** received from one or more other devices **110**. The second device/system **120** may aggregate **160** the model update data using various techniques to update the second model **125**. The second device/system **120** may generate **170** second model update data **175** for sending back to the first device **110**. The second device/system **120** may generate the second model update data **175** using a protocol the same as or similar to the protocol **150** used to generate the first model update data **155**. In some implementations, the second device/system **120** may send an indication of the protocol used to generate the second model update data **175** to the first device **110**. Example operations for aggregating **160** model updates and generating **170** model update data are described in additional detail below with reference to FIGS. **5**A and **5**B, respectively.

(28) Upon receiving the second model update data **175**, the first device **110** may process it using the appropriate protocol. The first device **110** may use the resulting data to update **180** the first model **115**. In some implementations, the first device **110** may perform one or more gatekeeping functions with regard to the second model update data **175**, such as validating and/or fine-tuning the model update data. For example, in the case of a speech processing system, an ASR model on the first device **110** may adapt to an accent and/or dialect of the user **5**. Thus, the first model **115**,

through the training **140**, may become adapted to the user's **5** speech. The second model update data **175** may, however, represent an aggregation of data possibly related to many users. It may be possible that the second model update data **175** may cause a dilution of refinements to the first model **115** particular to the user **5**. Therefore, in some implementations, the first device **110** may analyze the second model update data **175** and determine whether to accept or reject it. In some implementations, the first device **110** may perform a weighted update of the first model **115** where the first device **110** adjusts the first model **115** parameters by less than amounts indicated by the second model update data **175**; e.g., by one half, one quarter, or some other proportion.

(29) Following the update of the first model **115** based on the second model update data **175** received from the second device/system **120**, the process may repeat with the first device **110** continuing to train **140** the now updated first model **115** based on input data processed by the first device **110**.

(30) FIGS. **2**A-**2**D are conceptual diagrams illustrating various examples of sending and receiving secure updates for machine learning models, according to embodiments of the present disclosure. In each example, selection and reordering of first model **115** parameters may obscure the architecture and/or other aspects of the first model **115** in the event that a third part intercepts the first model update data **155** (and/or the second model update data **175**). A protocol **150** may be shared between the first device **110** and the second device/system **120** such that the second device/system **120** may be able to decode model update data received from the first device **110**, and vice versa. In some implementations, the first device **110** and/or the second device/system **120** may have a protocol storage (such as the protocol storage component **605** described below). In some implementations, the first device **110** and/or the second device/system **120** may send an indication of a particular protocol used to generate model update data to the recipient system/device. The recipient may then retrieve the indicated protocol **150** for decoding the model update data. In some implementations, the first device **110** and/or second device/system **120** may change the protocol occasionally; for example, for each model update or after a certain number of model updates. In various implementations, the first device **110** may use the same or different protocol(s) **150** to receive and/or decode model update data, and update **180** the first model **115**.

(31) FIG. **2**A illustrates a first example selection and transformation of first model **115** data to generate first model update data **155***a*. In this example, the first model **115** may include a plurality of layers L1-L5, and a protocol **150***a* may specify that the first model update data **155***a* is to include parameters from layers L3 and L4. The protocol **150***a* may further specify a transformation for the first model **115** parameters. In the example shown in FIG. **2**A, the transformation is a reordering of layer data from the first model **115** such that the layers L3 and L4 in the first model **115** appear in a different order within the first model update data **155***a*. The example shown in FIG. **2**A represents a reordering of two layers; however, more complicated reordering of a larger number of layers is possible.

(32) FIG. **2**B illustrates a second example selection and transformation of first model **115** data to generate first model update data **155***b*. Again, the first model **115** may include a plurality of layers L1-L5, and a protocol **150***b* may specify a random subset of the layers to include in the model update data. In the example shown in FIG. **2**B, the protocol **150***b* specifies that the first device **110** generate **145** the model update data using first model **115** layers L2 and L4. The layers L2 and/or L4 may be internal layers of the first model **115**. The protocol **150***b* may further specify that the layers L2 and L4 be included in the first model update data **155***b* transformed to be represented in the first model update data **155***b* as, for example, adjacent layers.

(33) FIG. **2**C illustrates a third example selection and transformation of first model **115** data to generate first model update data **155***c*. A protocol **150***c* may specify one or more mathematical operations for projecting first model **115** data into a different form. For example, the transformation may change the dimensions of a layer (e.g., a number of parameters per cell or per layer), change the number of layers, and/or values of the parameters. The transformation may result in

compression of the parameter data such that the first model update data **155***c* has a smaller size (e.g., in terms of megabytes) than the parameter data from the first model **115**. The operations specified by the protocol **150***c* may be reversible or partially reversible, such that the second device/system **120** may decode the first model update data **155***c* to determine the original first model **115** parameters (either exactly or approximately). In the example shown in FIG. **2**C, the protocol **150***c* specifies that the first device **110** include parameters from layers L2 and L4 of the first model **115** in the model update data. In addition to retrieving the parameters for layer L2 and L4 to generate **145** the model update data, the device may further project **245** the parameter data to determine the first model update data **155***c*. For example, the first device **110** may change the number of parameters per layer and/or obscure the layer location of the parameters.

(34) FIG. **2**D illustrates a fourth example selection and transformation of first model **115** data to generate first model update data **155***d*. Again, the first model **115** may include a plurality of layers L1-L5, and a protocol **150***d* may specify a subset of parameters from within one or more layers of the first model **115** to include in the model update data. In the example shown in FIG. **2**D, the protocol **150***d* specifies that the first device **110** generate **145** the model update data using only a portion of the parameters from each of the first model **115** layers L2 and L4.

(35) The operations shown in FIGS. **2**A through **2**D are provided as examples of various selection and transformation operations that may be specified by a protocol **150** for generating and decoding model update data. The different operations may be combined and/or repeated, and may extend to various machine learning architectures (e.g., including architectures that may not be organized into layers). In the example operations shown in FIGS. **2**A through **2**D, the second device/system **120** may receive the first model update data **155**, and decode it using the appropriate protocol **150** to determine the parameter values from the first model update data **155**. The second device/system **120** may then aggregate model update data from other first device **110** and update its own central model (e.g., the second model **125**). Examples of receiving, aggregating, and pushing model update data are described further below with reference to FIGS. **5**A and **5**B.

(36) FIG. **3**A is a flowchart describing example operations of a method **300** of a first device **110** generating, sending, and receiving secure updates for machine learning models, according to embodiments of the present disclosure. The method **300** may be performed by one or more devices **110** in communication with a system **120**.

(37) The method **300** may include operating (**305**) a first machine learning model to determine output data. The first device **110** may operate the machine learning model to, for example, recognize speech in audio data, objects and/or faces in image data, etc. Based on output data generated by the machine learning model (e.g., labels, predictions, inferences, etc.), the device **110** may update parameters of the machine learning model to improve its performance on subsequently received input data. From time to time (e.g., on demand, at predetermined times, after a predetermined number of updates, etc.) the device **110** may determine to send model update data to the system **120**, where the model update data represents some or all model parameters that have been updated.

(38) The method **300** may include determining (**310**) a first protocol for generating first model update data to be sent to a second device (e.g., the second device/system **120**). The first device **110** may determine the protocol in various ways. The device **110** may select a protocol at random, the device **110** may create the protocol based on a template and/or one or more arbitrary or random parameters, the device **110** may receive the protocol (or data indicating which protocol to use) from the system **120**, the device **110** may determine the protocol based on a schedule, etc. In some implementations, the device **110** may send an indication of the protocol in or accompanying the model update data. In some implementations, in addition or alternative to sending an indication of the protocol in the model update data, the system **120** may determine which protocol to use to decode the model update data based on a schedule shared with the device. For example, the system **120** may determine that model update data received during a certain time period corresponds to a

first protocol, and model update data received during a subsequent time period corresponds to a second protocol. The device **110** and/or system **120** may cycle through a finite number (e.g., 5, 10, 100, etc.) of protocols.

(39) The method **300** may include generating (**315**) the first model update data using a first subset of parameters of the first machine learning model and one or more operations indicated by the first protocol. The device **110** may generate the first model update data by determining a subset of machine learning model parameters to represent in the first model update data and/or transforming the subset of parameters; for example, to obscure parameter values and/or their correspondence to the structure of the machine learning model, and/or to reduce the size of the model update data relative to the size of the machine learning model (e.g., by including parameters from fewer than all layers of the machine learning model, and omitting parameters from other layers). The device **110** may transform the subset of parameters by taking a random subset of parameters less than the total number of parameters of the machine learning model, reordering the parameters, and/or projecting the parameters (e.g., as shown in FIGS. **2**A through **2**D).

(40) The method **300** may include sending (**320**), to the second device, the first model update data. In some implementations, the device **110** may send an indication of the protocol used to generate the first model update data (e.g., either in or accompanying the first model update data) to the second device (e.g., the system **120**). The second device may use the indication to retrieve the protocol, and use the protocol to decode the first model update data and determine the update parameters represented by the first model update data. The second device may use the updated parameters to update its own machine learning model. The second device may aggregate such updated parameters from multiple devices **110**, and its updated machine learning model to determine second model update data. The second device may send the second model update data back to the first device **110** (and, in some cases, other devices **110**).

(41) The method **300** may include receiving (**325**), from the second device in response to the first model update data, second model update data. The device **110** may determine a protocol for decoding the second model update data. In some implementations, the device **110** may process the second model update data using the same protocol used to generate the first model update data. In some implementations, the second model update data may include or be accompanied by a second indication of a second protocol. In any case, the device **110** may select and/or receive a second protocol for processing the second model update data.

(42) The method **300** may include determining (**330**), using the second model update data, a second subset of parameters. The device **110** may use the second protocol to decode the second model update data and determine the second subset of parameters. The method **300** may include generating (**335**), using the second subset, a second machine learning model representing a version of the first machine learning model updated based at least in part on second model update data.

(43) In some implementations, the device **110** may validate the second model update data. The device **110** may determine a dataset of samples (e.g., stored and/or simulated) of data of the type the machine learning model processes (e.g., audio data, images, sensor data, etc.). The device **110** may process the dataset using the first machine learning model and the second machine learning model. The device **110** may determine respective error rates of the output of each model. The device **110** may determine, based on the error rates, whether to process subsequent input data using the second machine learning model (e.g., if the update results in improved performance of the model such as a lower error rate) or to reject the model update and continue processing subsequent input data using the first machine learning model.

(44) In some implementations, the device **110** may perform a partial update; for example, by determining parameters for the second machine learning model by applying a weight factor to parameters of the first machine learning model and to parameters represented in the second model update data.

(45) The device **110** may repeat various stages of the method **300** to perform additional model

updates based on subsequently processed input data and/or model update data received from the system **120**.

(46) FIG. **3**B is a flowchart describing example operations of a method **350** of a second device/system **120** receiving, aggregating, and sending secure updates for machine learning models, according to embodiments of the present disclosure. The method **350** may be performed by a system **120** in communication with one or more devices **110**. The method **350** may include sending (**355**), to a first device operating a first machine learning model, an indication of a first protocol for generating first model update data, the first model update data representing an update of the first machine learning model. The method **350** may include causing (**360**) the first device to generate the first model update data using the first machine learning model and the first protocol. The method **350** may include receiving (**365**), from the first device, the first model update data. The method **350** may include receiving (**370**), from a second device, second model update data representing an update of a second machine learning model operated by the second device. The method **350** may include generating (**375**) third model update data using the first protocol, the first model update data, and the second model update data. The method **350** may include sending (**380**) the third model update data to the first device. The method **350** may include causing (**385**) the first device to generate a third machine learning model using the second protocol and the third model update data, the third machine learning model representing an update of the first machine learning model. The various stages of the method **350** may be repeated to perform additional model updates based on model update data subsequently received from the device **110** and/or other devices.

(47) FIG. **4** is a conceptual diagram illustrating example operations of a device training and updating a machine learning model, according to embodiments of the present disclosure. The first device **110** may train the first machine learning model **115** using the process **140**, which may include the steps **415** through **425** described below to generate the first model update data **155**. The first device **110** may send the first model update data **155** to the second device/system **160**, which may aggregate the first model update data **155** with additional model update data **165** received from other devices. The second device/system **120** may send the second model update data **175** (e.g., generated using the aggregated model update data) to the first device **110**. The first device **110** may update the first machine learning model **115** using the process **180**. The process of sending, receiving, aggregating, and implementing model updates may be repeated.

(48) The training process **140** may involve training the first machine learning model **115** using device data **405**. The device data **405** may represent one or more user interactions with the first device **110** and may include input and/or output data processed/generated by the first device **110** using the first machine learning model **115**. The device data **405** may include personal data such that that user settings and/or data privacy regulations may prohibit sending the device data **405** to other devices and/or systems. The device data **405** may, however, be used to train the first machine learning model **115** on the first device **110**. Based on the training, model update data representing the updated parameters may be shared with the second device/system **120** for updating the second machine learning model **125** and/or aggregating with other model update data.

(49) The training process **140** may include performing gradient descent and/or backpropagation algorithms to update parameters of some or all layers of the first machine learning model **115**. For example, when a subset of layers/parameters is trained, the other layers/parameters may be held constant. In various implementations, the training process **140** may include, for example, batch gradient descent, stochastic gradient descent, mini-batch gradient descent, etc., depending on use of the device data **405**. In a training iteration, the first device **110** may calculate 415 gradients for the parameters to be trained based on the device data **405** (or a subset of the device data **405**). Using the gradients calculated at the step **415** and a predetermined learning rate, the first device **110** may calculate 420 updated parameters for the first machine learning model **115**. The training process **140** may include one or more iterations per model update. Thus, the training process **140** may include determining **425** whether to iterate. If so (“yes” at **425**), the training process **140** may repeat

the steps **415** and **420**. If not ("no" at **425**), the training process **140** may end for the model update. The first device **110** may generate the first model update data **155** based on the updated parameters, and send the first model update data **155** to the second device/system **120**.

(50) FIG. **5**A is a conceptual diagram illustrating an example of aggregating model updates from multiple devices, according to embodiments of the present disclosure. In the example shown in FIG. **5**A, a first device **110-1** sends model update data **155-1** to the second device **120**. In addition, a third device **110-2** sends additional (third) model update data **155-2** to the second device **120**. The devices **110-1** and **110-2** may generate the model update data by one or more of the techniques described previously. The second device **120** may aggregate (**160**) the model update data using various mathematical operations including, for example, averaging, weighted averaging, and/or by using the model update data to update a second machine learning model maintained by the second device **120** (as shown in FIG. **5**B). The second device **120** may use the aggregated model update data to generate (**170**) the second model update data **175**. In some implementations, the second device **120** may determine updated layer parameters for sending back to the device **110-1** and/or **110-2**. In some implementations, the second device **120** may perform one or more operations on the updated layer parameters to obscure details of the machine learning models from would-be interceptors. The second device **120** may send the second model update data **175** back to the device **110-1** and/or the device **110-2**. The device **110-1** and/or **110-2** may use the second model update data **175** to update their own machine learning models.

(51) FIG. **5**B is a conceptual diagram illustrating a first example of using shared layer data from a first device **110** to update a model on a second device **120**, according to embodiments of the present disclosure. The first device **110** may operate a first machine learning model **115**, and generate first model update data **155** based on updates to one or more layers L1-L5 of the first machine learning model **115**. The second device **120** may operate a second machine learning model **125**, which may process a same type of data as the first machine learning model **115** (e.g., image data, audio data, etc.) and/or make similar inferences (e.g., object recognition, speech recognition, etc.). The first machine learning model **115** and the second machine learning model **125** may, however, have different architectures; for example, each may have a different number and/or type of layers, with layers having different numbers of cells and/or connections per cell, etc. The second device **120** may perform federated learning and/or knowledge distillation techniques to update the second model **125** based on the first model update data **155** (e.g., and without obtaining the data the first device **110** may have used to update the first model **115** to generate the first model update data **155**).

(52) In the example shown in FIG. **5**B, the first model update data **155** includes a representation of parameters of layers L2 and L4. The layers L2 and/or L4 may be internal layers of the first model **115**. The second device **120** may aggregate **510** the first model update data **155** into the second machine learning model **125**. In some implementations, the second device **120** may use a shared dataset **520** to bring the second model **125** (e.g., one or more layers L11-L17 of the second model **125**) into convergence with the parameters represented in the layers L2 and L4. The shared dataset **520** may include public data and/or synthesized data that includes samples of example data processed by the machine learning models. The shared dataset **520** may exclude any user data such as input data used by the first device **110** to update the first model **115**. In some implementations, the parameters of the Layers L2 and L4 may themselves be used as labels for training one or more corresponding layers L11-L17 of the second machine learning model **125**; that is, without using a shared dataset **520**.

(53) The second device **120** may bring the second model **125** into convergence with the layers L2 and L4. For example, the second device/system can estimate the Kullback-Leibler divergence and/or the mean-squared error between respective layers L11-L17 of the second model **125** and the layers L2 and L4. The layers of the respective models may generate intermediate information for each sample in the shared dataset **520**. The layers L11-L17 of the second model **125** may be trained

to bring the intermediate information into convergence with that of the layers L2 and/or L4. In some implementations, the second device **120** can bring one or more individual layers L11-L17 of the second model **125** into convergence with the layers L2 and L4. For example, the second device **120** may determine a correspondence **551** between the layer L2 and layers L11 through L13. The second device **120** may update parameters of the layers L11 through L13 such that the intermediate information **554***b* output by the layers L11 through L13 converges with the intermediate information **553***b* output by the layer L2. Similarly, the second device **120** may determine a correspondence **552** between the layer L4 and a single layer of the second machine learning model **125**, such as the layer L16. The layers L11-13 and/or L16 may be internal layers of the second model **125**. The second device **120** may update parameters of the layer L16 such that the intermediate information **554***a* output by the layer L16 converges with the intermediate information **553***a* output by the layer L4. In some implementations, the second device **120** may determine additional correspondences for additional shared layers, and update additional layers of the second model **125**. The updated second model **125** may be used to determine update shared layer parameters for generating the second model update data **175** for sending to the first device **110** and/or other devices.

(54) FIG. **5**C is a conceptual diagram illustrating a second example of using model update from a first device **110** to update a model on a second device/system **120**, according to embodiments of the present disclosure. The example shown in FIG. **5**C differs from that of FIG. **5**B in that the parameters of the layers L2 and L4 need not be shared between the first device **110** and the second device/system **120**. Rather, the first device **110** may input samples of a shared dataset **520***a* into the layers L2 and L4 to generate intermediate information **553***a* and **553***b*, respectively. The intermediate information may include one or more vectors generated based on each sample of a shared dataset **520***a*. The first device **110** may generate the first model update data **155** based on the intermediate information **553**, and send the intermediate information **553** to the second device/system **120** for use in updating the second model **125**.

(55) The second device/system **120** may process samples of a shared dataset **520***b* to generate intermediate information **554***a* and **554***b*. The samples may be the same or similar to the samples of the shared dataset **520***a* and, like the shared dataset **520**, include public, synthesized, and/or non-private information. The aggregation process **510** may use the intermediate information **553** and **554** to update the second model **125** to reflect updates to the first model **115** represented in the layers L2 and L4. In some cases, the aggregation process **510** may compare only the intermediate information **553***a* and **554***a*. As in the example, shown in FIG. **5**B, the second device **120** may bring the second model **125** into convergence with the layers L2 and L4 based the intermediate information **553**, but without receiving the layer L2 and L4 parameter values themselves. The second device/system **120** may estimate the Kullback-Leibler divergence and/or the mean-squared error between respective intermediate information **553** and **554**. The aggregation process **510** may adjust parameter values of the layers L11-L17 of the second model **125** to bring the intermediate information **554** into convergence with the intermediate information **553** received from the first device **110**. As in FIG. **5**B, the second device **120** may bring one or more individual layers L11-L17 of the second model **125** into convergence with the layers L2 and L4. For example, the second device **120** may determine a correspondence **551** between the layer L2 and layers L11 through L13. The second device **120** may update parameters of the layers L11 through L13 such that the intermediate information **554***b* output by the layers L11 through L13 converges with the intermediate information **553***b* output by the layer L2. Similarly, the second device **120** may determine a correspondence **552** between the layer L4 and a single layer of the second machine learning model **125**, such as the layer L16. The second device **120** may update parameters of the layer L16 such that the intermediate information **554***a* output by the layer L16 converges with the intermediate information **553***a* output by the layer L4. In some implementations, the second device **120** may determine additional correspondences for additional shared layers, and update additional

layers of the second model **125**. The updated second model **125** may be used to determine update shared layer parameters for generating the second model update data **175** for sending to the first device **110** and/or other devices.

(56) FIG. **5**D is a conceptual diagram illustrating an example of a second device/system **120** receiving a model update from a first device **110**, and aggregating the model update data into a second model **125**, according to embodiments of the present disclosure. The device **110** may generate model update data by one or more of the techniques described previously, and send the model update data to the system **120**. In the example shown in FIG. **5**D, the device **110** may send layers L2 and L4 of the first model **115**. The system **120** may receive the first model **115** layers and incorporate them into a reconstructed model **116**. The first model **116** may include the parameters of the layers L2 and L4 as provided by the device **110**. The system **120** may reconstruct the rest of the first model **116** by adding one or more additional layers (e.g., layers L1*, L3*, L5*, and/or others). The system **120** may populate the additional layers by one or more means, including using random parameter values, default parameter values, parameter values obtained from previous model update data from the device **110**, etc.

(57) The second device/system **120** may use the reconstructed model **116** to update the second model **125** using an aggregation process **510**, which may be repeated for additional models reconstructed from model update data received from other devices. The aggregation process **510** may be, for example, a type of knowledge distillation where the second model **125** is trained based on layers of the first model **115** (and vice-versa for generating the second model update data **175**). During the aggregation process **510**, the system **120** may input a shared dataset **520** into the reconstructed model **116** and the second model **125**, and iteratively adjust parameters of the second model **125** to bring it into convergence with the reconstructed model **116**. The shared dataset **520** may include public data and/or synthesized data that includes samples of example data processed by the machine learning models. The shared dataset **520** may exclude any user data such as input data used by the first device **110** to update the first model **115**. The shared dataset **520** may include a set of samples to be input into each model. For example, for a speech recognition model, the samples may be segments of stored or simulated audio data; similarly, for an image processing model, the sample may include still and/or video image data. The aggregation process **510** may compare output predictions and/or internal representations of the respective models. In some implementations, the parameters of the Layers L2 and L4 may themselves be used as labels for training one or more corresponding layers L11-L17 of the second machine learning model **125**; that is, without using a shared dataset **520**.

(58) For example, the system can estimate the Kullback-Leibler divergence and/or the mean-squared error between respective layers L11-L17 of the second model **125** and the layers reconstructed model **116** (and additional reconstructed models). The respective models may agree on labels generated for each sample in the shared dataset **520**. The second model **125** and reconstructed model **116** layer projections may be compared to measure whether models have similar information flow inside their layers (e.g., internal correspondence). The layers L11-L17 of the second model **125** may be trained to bring the internal correspondence into convergence with the reconstructed model **116**.

(59) Determining internal correspondence between two models is illustrated in FIG. **5**D. In addition to comparing output predictions (e.g., between the signals **555***a* and **555***b*), the system **120** can compare intermediate information between one or more layers (e.g., between the signals **553***a* and **553***b* and the signals **554***a* and **554***b*, respectively).

(60) In some implementations, and as illustrated in FIG. **5**D, the system **120** may aggregate model update data from a first model **115** into a second model **125** having an architecture different from the first model **115**. For example, the second model **125** may have a different number of layers and/or a different number of cells per layer than the first model **115**. In FIG. **5**D, the first model has five layers L1 through L5, and the second model **125** has seven layers L11 through L17.

Nevertheless, the second device/system **120** may train the second model **125** to, for example, produce a correspondence between the signal **554***b* output by the layer L13 and the signal **553***b* output by the layer L2; and similarly between the signal **554***a* output by the layer L16 and the signal **553***a* output by the layer L4, and between the signal **555***b* output by the layer L17 and the signal **555***a* output by the layer L5*. Thus, the aggregation process **510** may update one or more of the layers L11 through L17 of the second model **125**. In this manner, the system **120** may update the second model **125** based on updates from a first model **115** having a different architecture.

(61) FIG. **5**E is a conceptual diagram illustrating an example of generating model update data based on an updated second model **126**, and sending model update data to the first device **110**, according to embodiments of the present disclosure. The second model **125** may be replaced by the updated second model **126** based on the aggregation (e.g., using the aggregation process **510**) of model update data from one or more devices. A process **530** may be used to train an updated reconstructed model **117** based on the updated second model **126**. For example, the updated reconstructed model **117** may be generated based on obtaining a correspondence between output signals and/or input signals when processing samples in the shared dataset **520**. The second device/system **120** may determine a protocol for returning the model update data to the first device **110**. In the example shown in FIG. **5**E, the system **120** may return model update data for the updated layers L2′ and L4′. The system **120** may transform the data based on one or more of the techniques described previously, and send the model update data to the first device **110**. The first device **110** may receive the model update data, and decode it using the protocol to determine updated parameters for the first model **115**. The first model **115** may thus be replaced by the updated first model **118**, having updated layers L2′ and L4′.

(62) In some implementations, the first device **110** may analyze the received model update data and determine whether to accept or reject it. For example, the first device **110** may determine a shared dataset (e.g., the same as or different from the shared dataset **520** used by the system **120**), and determine respective error rate for the first model **115** and the updated first model **118**. Based on the determined error rates, the device **110** may determine to reject the updated first model **118** in favor of the previous first model **115**, or determine to accept the updated first model **118** and discard the previous first model **115**.

(63) In some implementations, the first device **110** may analyze the received model update data and determine to perform a partial update of the first model **115**. For example, the device **110** may process the shared dataset using both the first model **115** and the updated first model **118**. Each model may produce output data, which may include predictions, labels, and/or error rates. The first device **110** may determine, based on the respective output data, to fine-tune the received model update data. For example, the first device **110** may perform a weighted update of the subset of parameters of the first model **115** using a weighted average of the subset of parameters of the first model **115** and the subset of parameters determined from the second model update data.

(64) The processes described with reference to FIGS. **5**A through **5**E may repeat for future updates of the first model **115**/updated first model **118** based on further processing of input data.

(65) FIG. **6** is a conceptual diagram of components of the system **100**, according to embodiments of the present disclosure. The system may include components divided and/or shared between one or more devices **110**, systems **120**, and/or systems **625**. Some components may employ one or more machine learning models that may be updated based on processing input data using a first model, aggregating model update data from one or more devices **110** into a central model, and/or receiving update data from a system operating the central model. Components of the system **100** (e.g., the devices **110** and/or system **120**) may include a model update data generator component **601** for generating model update data based on the techniques described herein, a model updater component **603** for updating machine learning models based on received model update data, and a protocol storage component **605** for storing protocols used for generating the model update data based on updated model parameters, and decoding the model update data to determine the updated

model parameters.
(66) The system **100** may operate using various components as described in FIG. **6**. The various components may be located on same or different physical devices. Communication between various components may occur directly or across a network(s) **199**. The device **110** may include audio capture component(s), such as a microphone or array of microphones of a device **110**, captures audio **11** and creates corresponding audio data. Once speech is detected in audio data representing the audio **11**, the device **110** may determine if the speech is directed at the device **110**/system **120**. In at least some embodiments, such determination may be made using a wakeword detection component **620**. The wakeword detection component **620** may be configured to detect various wakewords. In at least some examples, each wakeword may correspond to a name of a different digital assistant. An example wakeword/digital assistant name is "Alexa." In another example, input to the system may be in form of text data **613**, for example as a result of a user typing an input into a user interface of device **110**. Other input forms may include indication that the user has pressed a physical or virtual button on device **110**, the user has made a gesture, etc. The device **110** may also capture images using camera(s) **1118** of the device **110**. The image data **621** may be used in various manners by different components of the device/system to perform operations such as determining whether a user is directing an utterance to the system, interpreting a user command, responding to a user command, etc.
(67) The wakeword detector **620** of the device **110** may process the audio data, representing the audio **11**, to determine whether speech is represented therein. The wakeword detector **620** may process the audio data using one or more models as described below. One or more of these models may be updated using the techniques discussed herein. The device **110** may use various techniques to determine whether the audio data includes speech. In some examples, the device **110** may apply voice-activity detection (VAD) techniques. Such techniques may determine whether speech is present in audio data based on various quantitative aspects of the audio data, such as the spectral slope between one or more frames of the audio data; the energy levels of the audio data in one or more spectral bands; the signal-to-noise ratios of the audio data in one or more spectral bands; or other quantitative aspects. In other examples, the device **110** may implement a classifier configured to distinguish speech from background noise. The classifier may be implemented by techniques such as linear classifiers, support vector machines, and decision trees. In still other examples, the device **110** may apply hidden Markov model (HMM) or Gaussian mixture model (GMM) techniques to compare the audio data to one or more acoustic models in storage, which acoustic models may include models corresponding to speech, noise (e.g., environmental noise or background noise), or silence. Still other techniques may be used to determine whether speech is present in audio data.
(68) Wakeword detection is typically performed without performing linguistic analysis, textual analysis, or semantic analysis. Instead, the audio data, representing the audio **11**, is analyzed to determine if specific characteristics of the audio data match preconfigured acoustic waveforms, audio signatures, or other data corresponding to a wakeword.
(69) Thus, the wakeword detection component **620** may compare audio data to stored data to detect a wakeword. One approach for wakeword detection applies general large vocabulary continuous speech recognition (LVCSR) systems to decode audio signals, with wakeword searching being conducted in the resulting lattices or confusion networks. Another approach for wakeword detection builds HMMs for each wakeword and non-wakeword speech signals, respectively. The non-wakeword speech includes other spoken words, background noise, etc. There can be one or more HMMs built to model the non-wakeword speech characteristics, which are named filler models. Viterbi decoding is used to search the best path in the decoding graph, and the decoding output is further processed to make the decision on wakeword presence. This approach can be extended to include discriminative information by incorporating a hybrid DNN-HMM decoding framework. In another example, the wakeword detection component **620** may be built on deep

neural network (DNN)/recursive neural network (RNN) structures directly, without HMM being involved. Such an architecture may estimate the posteriors of wakewords with context data, either by stacking frames within a context window for DNN, or using RNN. Follow-on posterior threshold tuning or smoothing is applied for decision making. Other techniques for wakeword detection, such as those known in the art, may also be used.

(70) Once the wakeword is detected by the wakeword detector **620** and/or input is detected by an input detector, the device **110** may "wake" and begin generating audio data **611**, representing the audio **11**. The device **110** may process the audio data **611** using one or more machine learning models described below. In some implementations, the first **110** device may send data representing a result of processing the audio data **611** and/or image data **621** to the second device/system(s) **120** for further processing. Audio data **611** sent to the second device/system **120** may include data corresponding to the wakeword; in other embodiments, the portion of the audio corresponding to the wakeword is removed by the device **110** prior to sending the audio data **611** to the system(s) **120**. In the case of touch input detection or gesture based input detection, the audio data may not include a wakeword.

(71) In some implementations, the system **100** may include more than one system **120**. The systems **120** may respond to different wakewords and/or perform different categories of tasks. Each system **120** may be associated with its own wakeword such that speaking a certain wakeword results in audio data be sent to and processed by a particular system. For example, detection of the wakeword "Alexa" by the wakeword detector **620** may result in sending data representing a user input to a first system **120** for processing while detection of the wakeword "Computer" by the wakeword detector may result in sending data representing a user input to a second system **120** for processing. The system may have a separate wakeword and system for different skills/systems (e.g., "Dungeon Master" for a game play skill/system **120**) and/or such skills/systems may be coordinated by one or more skill(s) **690***a*, **690***b*, and/or **690***c* (collectively, "skills **690**") of one or more systems **120**.

(72) Upon generation, the audio data **611** may be sent to an orchestrator component **630** of the first device **110** (and/or, in some implementations, the second device/system **120**). The orchestrator component **630** may include memory and logic that enables the orchestrator component **630** to transmit various pieces and forms of data to various components of the system, as well as perform other operations as described herein.

(73) The orchestrator component **630** may send the audio data **611** to a language processing component **692**. The language processing component **692** (sometimes also referred to as a spoken language understanding (SLU) component) includes an automatic speech recognition (ASR) component **650** and a natural language understanding (NLU) component **660**. The ASR component **650** may transcribe the audio data **611** into text data. The text data output by the ASR component **650** represents one or more than one (e.g., in the form of an N-best list) ASR hypotheses representing speech represented in the audio data **611**. The ASR component **650** interprets the speech in the audio data **611** based on a similarity between the audio data **611** and pre-established language models. For example, the ASR component **650** may compare the audio data **611** with models for sounds (e.g., acoustic units such as phonemes, senons, phones, etc.) and sequences of sounds to identify words that match the sequence of sounds of the speech represented in the audio data **611**. The ASR component **650** sends the text data generated thereby to an NLU component **660**, via, in some embodiments, the orchestrator component **630**. The text data sent from the ASR component **650** to the NLU component **660** may include a single top-scoring ASR hypothesis or may include an N-best list including multiple top-scoring ASR hypotheses. An N-best list may additionally include a respective score associated with each ASR hypothesis represented therein. The ASR component **650** is described in greater detail below with regard to FIG. **7**.

(74) The speech processing system **692** may further include a NLU component **660**. The NLU component **660** may receive the text data from the ASR component. The NLU component **660** may attempts to make a semantic interpretation of the phrase(s) or statement(s) represented in the text

data input therein by determining one or more meanings associated with the phrase(s) or statement(s) represented in the text data. The NLU component **660** may determine an intent representing an action that a user desires be performed and may determine information that allows a device (e.g., the device **110**, the system(s) **120**, a skill component **690**, a skill system(s) **625**, etc.) to execute the intent. For example, if the text data corresponds to "play the 5.sup.th Symphony by Beethoven," the NLU component **660** may determine an intent that the system output music and may identify "Beethoven" as an artist/composer and "5th Symphony" as the piece of music to be played. For further example, if the text data corresponds to "what is the weather," the NLU component **660** may determine an intent that the system output weather information associated with a geographic location of the device **110**. In another example, if the text data corresponds to "turn off the lights," the NLU component **660** may determine an intent that the system turn off lights associated with the device **110** or the user **5**. However, if the NLU component **660** is unable to resolve the entity—for example, because the entity is referred to by anaphora such as "this song" or "my next appointment"—the speech processing system **692** can send a decode request to another speech processing system **692** for information regarding the entity mention and/or other context related to the utterance. The speech processing system **692** may augment, correct, or base results data upon the audio data **611** as well as any data received from the other speech processing system **692**.

(75) The NLU component **660** may return NLU results data (which may include tagged text data, indicators of intent, etc.) back to the orchestrator **630**. The orchestrator **630** may forward the NLU results data to a skill component(s) **690**. If the NLU results data includes a single NLU hypothesis, the NLU component **660** and the orchestrator component **630** may direct the NLU results data to the skill component(s) **690** associated with the NLU hypothesis. If the NLU results data includes an N-best list of NLU hypotheses, the NLU component **660** and the orchestrator component **630** may direct the top scoring NLU hypothesis to a skill component(s) **690** associated with the top scoring NLU hypothesis. The system may also include a post-NLU ranker which may incorporate other information to rank potential interpretations determined by the NLU component **660**. The device **110** may also include its own post-NLU ranker. The NLU component **660**, post-NLU ranker, and other components are described in greater detail below with regard to FIG. **8**.

(76) A skill component may be software running on the first device **110** and/or second device/system(s) **120** that is akin to a software application. That is, a skill component **690** may enable the devices(s) **110**/**120** to execute specific functionality in order to provide data or produce some other requested output. As used herein, a "skill component" may refer to software that may be placed on a machine or a virtual machine (e.g., software that may be launched in a virtual instance when called). A skill component may be software customized to perform one or more actions as indicated by a business entity, device manufacturer, user, etc. What is described herein as a skill component may be referred to using many different terms, such as an action, bot, app, or the like. The devices(s) **110**/**120** may be configured with more than one skill component **690**. For example, a weather service skill component may enable the devices(s) **110**/**120** to provide weather information, a car service skill component may enable the devices(s) **110**/**120** to book a trip with respect to a taxi or ride sharing service, a restaurant skill component may enable the devices(s) **110**/**120** to order a pizza with respect to the restaurant's online ordering system, etc. A skill component **690** may operate in conjunction between the devices(s) **110**/**120** and other devices, such as the device **110**, in order to complete certain functions. Inputs to a skill component **690** may come from speech processing interactions or through other interactions or input sources. A skill component **690** may include hardware, software, firmware, or the like that may be dedicated to a particular skill component **690** or shared among different skill components **690**.

(77) A skill support system(s) **625** may communicate with a skill component(s) **690** within the devices(s) **110**/**120** and/or directly with the orchestrator component **630** or with other components. A skill support system(s) **625** may be configured to perform one or more actions. An ability to

perform such action(s) may sometimes be referred to as a “skill.” That is, a skill may enable a skill support system(s) **625** to execute specific functionality in order to provide data or perform some other action requested by a user. For example, a weather service skill may enable a skill support system(s) **625** to provide weather information to the devices(s) **110**/**120**, a car service skill may enable a skill support system(s) **625** to book a trip with respect to a taxi or ride sharing service, an order pizza skill may enable a skill support system(s) **625** to order a pizza with respect to a restaurant's online ordering system, etc. Additional types of skills include home automation skills (e.g., skills that enable a user to control home devices such as lights, door locks, cameras, thermostats, etc.), entertainment device skills (e.g., skills that enable a user to control entertainment devices such as smart televisions), video skills, flash briefing skills, as well as custom skills that are not associated with any pre-configured type of skill.

(78) The devices(s) **110**/**120** may be configured with a skill component **690** dedicated to interacting with the skill support system(s) **625**. Unless expressly stated otherwise, reference to a skill, skill device, or skill component may include a skill component **690** operated by the devices(s) **110**/**120** and/or skill operated by the skill support system(s) **625**. Moreover, the functionality described herein as a skill or skill may be referred to using many different terms, such as an action, bot, app, or the like. The skill **690** and or skill support system(s) **625** may return output data to the orchestrator **630**.

(79) The devices(s) **110**/**120** includes a language output component **693**. The language output component **693** includes a natural language generation (NLG) component **679** and a text-to-speech (TTS) component **680**. The NLG component **679** can generate text for purposes of TTS output to a user. For example the NLG component **679** may generate text corresponding to instructions corresponding to a particular action for the user to perform. The NLG component **679** may generate appropriate text for various outputs as described herein. The NLG component **679** may include one or more trained models configured to output text appropriate for a particular input. The text output by the NLG component **679** may become input for the TTS component **680** (e.g., output text data **910** discussed below). Alternatively or in addition, the TTS component **680** may receive text data from a skill **690** or other system component for output.

(80) The NLG component **679** may include a trained model. The NLG component **679** generates text data **910** from dialog data (e.g., received from a dialog manager) such that the output text data **910** has a natural feel and, in some embodiments, includes words and/or phrases specifically formatted for a requesting individual. The NLG may use templates to formulate responses. And/or the NLG system may include models trained from the various templates for forming the output text data **910**. For example, the NLG system may analyze transcripts of local news programs, television shows, sporting events, or any other media program to obtain common components of a relevant language and/or region. As one illustrative example, the NLG system may analyze a transcription of a regional sports program to determine commonly used words or phrases for describing scores or other sporting news for a particular region. The NLG may further receive, as inputs, a dialog history, an indicator of a level of formality, and/or a command history or other user history such as the dialog history.

(81) The NLG system may generate dialog data based on one or more response templates. Further continuing the example above, the NLG system may select a template in response to the question, “What is the weather currently like?” of the form: “The weather currently is $weather_information$.” The NLG system may analyze the logical form of the template to produce one or more textual responses including markups and annotations to familiarize the response that is generated. In some embodiments, the NLG system may determine which response is the most appropriate response to be selected. The selection may, therefore, be based on past responses, past questions, a level of formality, and/or any other feature, or any other combination thereof. Responsive audio data representing the response generated by the NLG system may then be generated using the text-to-speech component **680**.

(82) The TTS component **680** may generate audio data (e.g., synthesized speech) from text data using one or more different methods. Text data input to the TTS component **680** may come from a skill component **690**, the orchestrator component **630**, or another component of the system. In one method of synthesis called unit selection, the TTS component **680** matches text data against a database of recorded speech. The TTS component **680** selects matching units of recorded speech and concatenates the units together to form audio data. In another method of synthesis called parametric synthesis, the TTS component **680** varies parameters such as frequency, volume, and noise to create audio data including an artificial speech waveform. Parametric synthesis uses a computerized voice generator, sometimes called a vocoder.
(83) The device **110** may include still image and/or video capture components such as a camera or cameras to capture one or more images. The device **110** may include circuitry for digitizing the images and/or video for transmission to the system(s) **120** as image data or other representational data. The device **110** may further include circuitry for voice command-based control of the camera, allowing a user **5** to request capture of image or video data. The device **110** may process the commands locally or send audio data **611** representing the commands to the system(s) **120** for processing, after which the system(s) **120** may return output data that can cause the device **110** to engage its camera.
(84) Upon generation by the first device **110** and/or receipt by the second device/system(s) **120**, the image data **621** may be sent to an orchestrator component **630**. The orchestrator component **630** may send the image data **621** to an image processing component **640**. The image processing component **640** can perform computer vision functions such as object recognition, modeling, reconstruction, etc. For example, the image processing component **640** may detect a person, face, etc. The image processing component **640** is described in greater detail below with regard to FIG. **10**. The device **110** may also include an image processing component **640**.
(85) In some implementations, the image processing component **640** can detect the presence of text in an image. In such implementations, the image processing component **640** can recognize the presence of text, convert the image data to text data, and send the resulting text data via the orchestrator component **630** to the language processing component **692** for processing by the NLU component **660**.
(86) The system **100** (either on device **110**, system **120**, or a combination thereof) may include profile storage **670** for storing a variety of information related to individual users, groups of users, devices, etc. that interact with the system. As used herein, a “profile” refers to a set of data associated with a user, group of users, device, etc. The data of a profile may include preferences specific to the user, device, etc.; input and output capabilities of the device; internet connectivity information; user bibliographic information; subscription information, as well as other information.
(87) The profile storage **670** may include one or more user profiles, with each user profile being associated with a different user identifier/user profile identifier. Each user profile may include various user identifying data. Each user profile may also include data corresponding to preferences of the user. Each user profile may also include preferences of the user and/or one or more device identifiers, representing one or more devices of the user. For instance, the user account may include one or more IP addresses, MAC addresses, and/or device identifiers, such as a serial number, of each additional electronic device associated with the identified user account. When a user logs into to an application installed on a device **110**, the user profile (associated with the presented login information) may be updated to include information about the device **110**, for example with an indication that the device is currently in use. Each user profile may include identifiers of skills that the user has enabled. When a user enables a skill, the user is providing the devices(s) **110**/**120** with permission to allow the skill to execute with respect to the user's natural language user inputs. If a user does not enable a skill, the devices(s) **110**/**120** may not invoke the skill to execute with respect to the user's natural language user inputs.
(88) The profile storage **670** may include one or more group profiles. Each group profile may be

associated with a different group identifier. A group profile may be specific to a group of users. That is, a group profile may be associated with two or more individual user profiles. For example, a group profile may be a household profile that is associated with user profiles associated with multiple users of a single household. A group profile may include preferences shared by all the user profiles associated therewith. Each user profile associated with a group profile may additionally include preferences specific to the user associated therewith. That is, each user profile may include preferences unique from one or more other user profiles associated with the same group profile. A user profile may be a stand-alone profile or may be associated with a group profile.

(89) The profile storage **670** may include one or more device profiles. Each device profile may be associated with a different device identifier. Each device profile may include various device identifying information. Each device profile may also include one or more user identifiers, representing one or more users associated with the device. For example, a household device's profile may include the user identifiers of users of the household.

(90) FIG. **7** is a conceptual diagram of an ASR component **650**, according to embodiments of the present disclosure. The ASR component **650** may process the audio data to determine spoken language therein using one or more models as described below. The models may be updated by the techniques described herein. The device(s) **110** and/or system(s) **120** may include the ASR component **650**. The ASR component **650** may located across different physical and/or virtual machines. The ASR component **650** may interpret a spoken natural language input based on the similarity between the spoken natural language input and pre-established language models **754** stored in an ASR model storage **752**. For example, the ASR component **650** may compare the audio data with models for sounds (e.g., subword units or phonemes) and sequences of sounds to identify words that match the sequence of sounds spoken in the natural language input. Alternatively, the ASR component **650** may use a finite state transducer (FST) **755** to implement the language model functions.

(91) When the ASR component **650** generates more than one ASR hypothesis for a single spoken natural language input, each ASR hypothesis may be assigned a score (e.g., probability score, confidence score, etc.) representing a likelihood that the corresponding ASR hypothesis matches the spoken natural language input (e.g., representing a likelihood that a particular set of words matches those spoken in the natural language input). The score may be based on a number of factors including, for example, the similarity of the sound in the spoken natural language input to models for language sounds (e.g., an acoustic model **753** stored in the ASR model storage **752**), and the likelihood that a particular word, which matches the sounds, would be included in the sentence at the specific location (e.g., using a language or grammar model **754**). Based on the considered factors and the assigned confidence score, the ASR component **650** may output an ASR hypothesis that most likely matches the spoken natural language input, or may output multiple ASR hypotheses in the form of a lattice or an N-best list, with each ASR hypothesis corresponding to a respective score.

(92) The ASR component **650** may include a speech recognition engine **758**. The ASR component **650** receives audio data **611** (for example, received from a device **110** having processed audio detected by a microphone by an acoustic front end (AFE) or other component). The speech recognition engine **758** compares the audio data **611** with acoustic models **753**, language models **754**, FST(s) **755**, and/or other data models and information for recognizing the speech conveyed in the audio data. The audio data **611** may be audio data that has been digitized (for example by an AFE) into frames representing time intervals for which the AFE determines a number of values, called features, representing the qualities of the audio data, along with a set of those values, called a feature vector, representing the features/qualities of the audio data within the frame. In at least some embodiments, audio frames may be 10 ms each. Many different features may be determined, as known in the art, and each feature may represent some quality of the audio that may be useful for ASR processing. A number of approaches may be used by an AFE to process the audio data,

such as mel-frequency cepstral coefficients (MFCCs), perceptual linear predictive (PLP) techniques, neural network feature vector techniques, linear discriminant analysis, semi-tied covariance matrices, or other approaches known to those of skill in the art.

(93) The speech recognition engine **758** may process the audio data **611** with reference to information stored in the ASR model storage **752**. Feature vectors of the audio data **611** may arrive at the system **120** encoded, in which case they may be decoded prior to processing by the speech recognition engine **758**.

(94) The speech recognition engine **758** attempts to match received feature vectors to language acoustic units (e.g., phonemes) and words as known in the stored acoustic models **753**, language models 6B54, and FST(s) **755**. For example, audio data **611** may be processed by one or more acoustic model(s) **753** to determine acoustic unit data. The acoustic unit data may include indicators of acoustic units detected in the audio data **611** by the ASR component **650**. For example, acoustic units can consist of one or more of phonemes, diaphonemes, tonemes, phones, diphones, triphones, or the like. The acoustic unit data can be represented using one or a series of symbols from a phonetic alphabet such as the X-SAMPA, the International Phonetic Alphabet, or Initial Teaching Alphabet (ITA) phonetic alphabets. In some implementations a phoneme representation of the audio data can be analyzed using an n-gram based tokenizer. An entity, or a slot representing one or more entities, can be represented by a series of n-grams.

(95) The acoustic unit data may be processed using the language model **754** (and/or using FST **755**) to determine ASR data **710**. The ASR data **710** can include one or more hypotheses. One or more of the hypotheses represented in the ASR data **710** may then be sent to further components (such as the NLU component **660**) for further processing as discussed herein. The ASR data **710** may include representations of text of an utterance, such as words, subword units, or the like.

(96) The speech recognition engine **758** computes scores for the feature vectors based on acoustic information and language information. The acoustic information (such as identifiers for acoustic units and/or corresponding scores) is used to calculate an acoustic score representing a likelihood that the intended sound represented by a group of feature vectors matches a language phoneme. The language information is used to adjust the acoustic score by considering what sounds and/or words are used in context with each other, thereby improving the likelihood that the ASR component **650** will output ASR hypotheses that make sense grammatically. The specific models used may be general models or may be models corresponding to a particular domain, such as music, banking, etc.

(97) The speech recognition engine **758** may use a number of techniques to match feature vectors to phonemes, for example using Hidden Markov Models (HMMs) to determine probabilities that feature vectors may match phonemes. Sounds received may be represented as paths between states of the HMM and multiple paths may represent multiple possible text matches for the same sound. Further techniques, such as using FSTs, may also be used.

(98) The speech recognition engine **758** may use the acoustic model(s) **753** to attempt to match received audio feature vectors to words or subword acoustic units. An acoustic unit may be a senone, phoneme, phoneme in context, syllable, part of a syllable, syllable in context, or any other such portion of a word. The speech recognition engine **758** computes recognition scores for the feature vectors based on acoustic information and language information. The acoustic information is used to calculate an acoustic score representing a likelihood that the intended sound represented by a group of feature vectors match a subword unit. The language information is used to adjust the acoustic score by considering what sounds and/or words are used in context with each other, thereby improving the likelihood that the ASR component **650** outputs ASR hypotheses that make sense grammatically.

(99) The speech recognition engine **758** may use a number of techniques to match feature vectors to phonemes or other acoustic units, such as diphones, triphones, etc. One common technique is using Hidden Markov Models (HMMs). HMMs are used to determine probabilities that feature

vectors may match phonemes. Using HMMs, a number of states are presented, in which the states together represent a potential phoneme (or other acoustic unit, such as a triphone) and each state is associated with a model, such as a Gaussian mixture model or a deep belief network. Transitions between states may also have an associated probability, representing a likelihood that a current state may be reached from a previous state. Sounds received may be represented as paths between states of the HMM and multiple paths may represent multiple possible text matches for the same sound. Each phoneme may be represented by multiple potential states corresponding to different known pronunciations of the phonemes and their parts (such as the beginning, middle, and end of a spoken language sound). An initial determination of a probability of a potential phoneme may be associated with one state. As new feature vectors are processed by the speech recognition engine **758**, the state may change or stay the same, based on the processing of the new feature vectors. A Viterbi algorithm may be used to find the most likely sequence of states based on the processed feature vectors.

(100) The probable phonemes and related states/state transitions, for example HMM states, may be formed into paths traversing a lattice of potential phonemes. Each path represents a progression of phonemes that potentially match the audio data represented by the feature vectors. One path may overlap with one or more other paths depending on the recognition scores calculated for each phoneme. Certain probabilities are associated with each transition from state to state. A cumulative path score may also be calculated for each path. This process of determining scores based on the feature vectors may be called acoustic modeling. When combining scores as part of the ASR processing, scores may be multiplied together (or combined in other ways) to reach a desired combined score or probabilities may be converted to the log domain and added to assist processing.

(101) The speech recognition engine **758** may also compute scores of branches of the paths based on language models or grammars. Language modeling involves determining scores for what words are likely to be used together to form coherent words and sentences. Application of a language model may improve the likelihood that the ASR component **650** correctly interprets the speech contained in the audio data. For example, for an input audio sounding like “hello,” acoustic model processing that returns the potential phoneme paths of “H E L O”, “H A L O”, and “Y E L O” may be adjusted by a language model to adjust the recognition scores of “H E L O” (interpreted as the word “hello”), “H A L O” (interpreted as the word “halo”), and “Y E L O” (interpreted as the word “yellow”) based on the language context of each word within the spoken utterance.

(102) FIG. **8** is a conceptual diagram of how natural language processing is performed, according to embodiments of the present disclosure. The NLU component **660** may process the text data to determine semantic meaning using one or more models as described below. The models may be updated by the techniques described herein. The device(s) **110** and/or system(s) **120** may include the NLU component **660**. The NLU component **660** may located across different physical and/or virtual machines. FIG. **8** illustrates how NLU processing is performed on text data. The NLU component **660** may process text data including several ASR hypotheses of a single user input. For example, if the ASR component **650** outputs text data including an n-best list of ASR hypotheses, the NLU component **660** may process the text data with respect to all (or a portion of) the ASR hypotheses represented therein.

(103) The NLU component **660** may annotate text data by parsing and/or tagging the text data. For example, for the text data “tell me the weather for Seattle,” the NLU component **660** may tag “tell me the weather for Seattle” as an <OutputWeather> intent as well as separately tag “Seattle” as a location for the weather information.

(104) The NLU component **660** may include a shortlister component **850**. The shortlister component **850** selects skills that may execute with respect to ASR output data **710** input to the NLU component **660** (e.g., applications that may execute with respect to the user input). The ASR output data **710** (which may also be referred to as ASR data **710**) may include representations of text of an utterance, such as words, subword units, or the like. The shortlister component **850** thus

limits downstream, more resource intensive NLU processes to being performed with respect to skills that may execute with respect to the user input.
(105) Without a shortlister component **850**, the NLU component **660** may process ASR output data **710** input thereto with respect to every skill of the system, either in parallel, in series, or using some combination thereof. By implementing a shortlister component **850**, the NLU component **660** may process ASR output data **710** with respect to only the skills that may execute with respect to the user input. This reduces total compute power and latency attributed to NLU processing.
(106) The shortlister component **850** may include one or more trained models. The model(s) may be trained to recognize various forms of user inputs that may be received by the system(s) **120**. For example, during a training period skill system(s) **625** associated with a skill may provide the system(s) **120** with training text data representing sample user inputs that may be provided by a user to invoke the skill. For example, for a ride sharing skill, a skill system(s) **625** associated with the ride sharing skill may provide the system(s) **120** with training text data including text corresponding to "get me a cab to [location]," "get me a ride to [location]," "book me a cab to [location]," "book me a ride to [location]," etc. The one or more trained models that will be used by the shortlister component **850** may be trained, using the training text data representing sample user inputs, to determine other potentially related user input structures that users may try to use to invoke the particular skill. During training, the system(s) **120** may solicit the skill system(s) **625** associated with the skill regarding whether the determined other user input structures are permissible, from the perspective of the skill system(s) **625**, to be used to invoke the skill. The alternate user input structures may be derived by one or more trained models during model training and/or may be based on user input structures provided by different skills. The skill system(s) **625** associated with a particular skill may also provide the system(s) **120** with training text data indicating grammar and annotations. The system(s) **120** may use the training text data representing the sample user inputs, the determined related user input(s), the grammar, and the annotations to train a model(s) that indicates when a user input is likely to be directed to/handled by a skill, based at least in part on the structure of the user input. Each trained model of the shortlister component **850** may be trained with respect to a different skill. Alternatively, the shortlister component **850** may use one trained model per domain, such as one trained model for skills associated with a weather domain, one trained model for skills associated with a ride sharing domain, etc.
(107) The system(s) **120** may use the sample user inputs provided by a skill system(s) **625**, and related sample user inputs potentially determined during training, as binary examples to train a model associated with a skill associated with the skill system(s) **625**. The model associated with the particular skill may then be operated at runtime by the shortlister component **850**. For example, some sample user inputs may be positive examples (e.g., user inputs that may be used to invoke the skill). Other sample user inputs may be negative examples (e.g., user inputs that may not be used to invoke the skill).
(108) As described above, the shortlister component **850** may include a different trained model for each skill of the system, a different trained model for each domain, or some other combination of trained model(s). For example, the shortlister component **850** may alternatively include a single model. The single model may include a portion trained with respect to characteristics (e.g., semantic characteristics) shared by all skills of the system. The single model may also include skill-specific portions, with each skill-specific portion being trained with respect to a specific skill of the system. Implementing a single model with skill-specific portions may result in less latency than implementing a different trained model for each skill because the single model with skill-specific portions limits the number of characteristics processed on a per skill level.
(109) The portion trained with respect to characteristics shared by more than one skill may be clustered based on domain. For example, a first portion of the portion trained with respect to multiple skills may be trained with respect to weather domain skills, a second portion of the portion trained with respect to multiple skills may be trained with respect to music domain skills, a third

portion of the portion trained with respect to multiple skills may be trained with respect to travel domain skills, etc.

(110) Clustering may not be beneficial in every instance because it may cause the shortlister component **850** to output indications of only a portion of the skills that the ASR output data **710** may relate to. For example, a user input may correspond to “tell me about Tom Collins.” If the model is clustered based on domain, the shortlister component **850** may determine the user input corresponds to a recipe skill (e.g., a drink recipe) even though the user input may also correspond to an information skill (e.g., including information about a person named Tom Collins).

(111) The NLU component **660** may include one or more recognizers **863**. In at least some embodiments, a recognizer **863** may be associated with a skill system **625** (e.g., the recognizer may be configured to interpret text data to correspond to the skill system **625**). In at least some other examples, a recognizer **863** may be associated with a domain such as smart home, video, music, weather, custom, etc. (e.g., the recognizer may be configured to interpret text data to correspond to the domain).

(112) If the shortlister component **850** determines ASR output data **710** is potentially associated with multiple domains, the recognizers **863** associated with the domains may process the ASR output data **710**, while recognizers **863** not indicated in the shortlister component **850**'s output may not process the ASR output data **710**. The “shortlisted” recognizers **863** may process the ASR output data **710** in parallel, in series, partially in parallel, etc. For example, if ASR output data **710** potentially relates to both a communications domain and a music domain, a recognizer associated with the communications domain may process the ASR output data **710** in parallel, or partially in parallel, with a recognizer associated with the music domain processing the ASR output data **710**.

(113) Each recognizer **863** may include a named entity recognition (NER) component **862**. The NER component **862** attempts to identify grammars and lexical information that may be used to construe meaning with respect to text data input therein. The NER component **862** identifies portions of text data that correspond to a named entity associated with a domain, associated with the recognizer **863** implementing the NER component **862**. The NER component **862** (or other component of the NLU component **660**) may also determine whether a word refers to an entity whose identity is not explicitly mentioned in the text data, for example “him,” “her,” “it” or other anaphora, exophora, or the like.

(114) Each recognizer **863**, and more specifically each NER component **862**, may be associated with a particular grammar database **876**, a particular set of intents/actions **874**, and a particular personalized lexicon **886**. The grammar databases **876**, and intents/actions **874** may be stored in an NLU storage **873**. Each gazetteer **884** may include domain/skill-indexed lexical information associated with a particular user and/or device **110**. For example, a Gazetteer A (**884***a*) includes skill-indexed lexical information **886***aa* to **886***an*. A user's music domain lexical information might include album titles, artist names, and song names, for example, whereas a user's communications domain lexical information might include the names of contacts. Since every user's music collection and contact list is presumably different. This personalized information improves later performed entity resolution.

(115) An NER component **862** applies grammar information **876** and lexical information **886** associated with a domain (associated with the recognizer **863** implementing the NER component **862**) to determine a mention of one or more entities in text data. In this manner, the NER component **862** identifies “slots” (each corresponding to one or more particular words in text data) that may be useful for later processing. The NER component **862** may also label each slot with a type (e.g., noun, place, city, artist name, song name, etc.).

(116) Each grammar database **876** includes the names of entities (i.e., nouns) commonly found in speech about the particular domain to which the grammar database **876** relates, whereas the lexical information **886** is personalized to the user and/or the device **110** from which the user input originated. For example, a grammar database **876** associated with a shopping domain may include a

database of words commonly used when people discuss shopping.
(117) A downstream process called entity resolution (discussed in detail elsewhere herein) links a slot of text data to a specific entity known to the system. To perform entity resolution, the NLU component **660** may utilize gazetteer information (**884***a*-**884***n*) stored in an entity library storage **882**. The gazetteer information **884** may be used to match text data (representing a portion of the user input) with text data representing known entities, such as song titles, contact names, etc. Gazetteers **884** may be linked to users (e.g., a particular gazetteer may be associated with a specific user's music collection), may be linked to certain domains (e.g., a shopping domain, a music domain, a video domain, etc.), or may be organized in a variety of other ways.
(118) Each recognizer **863** may also include an intent classification (IC) component **864**. An IC component **864** parses text data to determine an intent(s) (associated with the domain associated with the recognizer **863** implementing the IC component **864**) that potentially represents the user input. An intent represents to an action a user desires be performed. An IC component **864** may communicate with a database **874** of words linked to intents. For example, a music intent database may link words and phrases such as “quiet,” “volume off,” and “mute” to a <Mute> intent. An IC component **864** identifies potential intents by comparing words and phrases in text data (representing at least a portion of the user input) to the words and phrases in an intents database **874** (associated with the domain that is associated with the recognizer **863** implementing the IC component **864**).
(119) The intents identifiable by a specific IC component **864** are linked to domain-specific (i.e., the domain associated with the recognizer **863** implementing the IC component **864**) grammar frameworks **876** with “slots” to be filled. Each slot of a grammar framework **876** corresponds to a portion of text data that the system believes corresponds to an entity. For example, a grammar framework **876** corresponding to a <PlayMusic> intent may correspond to text data sentence structures such as “Play {Artist Name},” “Play {Album Name},” “Play {Song name},” “Play {Song name} by {Artist Name},” etc. However, to make entity resolution more flexible, grammar frameworks **876** may not be structured as sentences, but rather based on associating slots with grammatical tags.
(120) For example, an NER component **862** may parse text data to identify words as subject, object, verb, preposition, etc. based on grammar rules and/or models prior to recognizing named entities in the text data. An IC component **864** (implemented by the same recognizer **863** as the NER component **862**) may use the identified verb to identify an intent. The NER component **862** may then determine a grammar model **876** associated with the identified intent. For example, a grammar model **876** for an intent corresponding to <PlayMusic> may specify a list of slots applicable to play the identified “object” and any object modifier (e.g., a prepositional phrase), such as {Artist Name}, {Album Name}, {Song name}, etc. The NER component **862** may then search corresponding fields in a lexicon **886** (associated with the domain associated with the recognizer **863** implementing the NER component **862**), attempting to match words and phrases in text data the NER component **862** previously tagged as a grammatical object or object modifier with those identified in the lexicon **886**.
(121) An NER component **862** may perform semantic tagging, which is the labeling of a word or combination of words according to their type/semantic meaning. An NER component **862** may parse text data using heuristic grammar rules, or a model may be constructed using techniques such as Hidden Markov Models, maximum entropy models, log linear models, conditional random fields (CRF), and the like. For example, an NER component **862** implemented by a music domain recognizer may parse and tag text data corresponding to “play mother's little helper by the rolling stones” as {Verb}: “Play,” {Object}: “mother's little helper,” {Object Preposition}: “by,” and {Object Modifier}: “the rolling stones.” The NER component **862** identifies “Play” as a verb based on a word database associated with the music domain, which an IC component **864** (also implemented by the music domain recognizer) may determine corresponds to a <PlayMusic>

intent. At this stage, no determination has been made as to the meaning of “mother's little helper” or “the rolling stones,” but based on grammar rules and models, the NER component **862** has determined the text of these phrases relates to the grammatical object (i.e., entity) of the user input represented in the text data.

(122) An NER component **862** may tag text data to attribute meaning thereto. For example, an NER component **862** may tag “play mother's little helper by the rolling stones” as: {domain} Music, {intent} <PlayMusic>, {artist name} rolling stones, {media type} SONG, and {song title} mother's little helper. For further example, the NER component **862** may tag “play songs by the rolling stones” as: {domain} Music, {intent} <PlayMusic>, {artist name} rolling stones, and {media type} SONG.

(123) The shortlister component **850** may receive ASR output data **710** output from the ASR component **650** or output from the device **110**. The ASR component **650** may embed the ASR output data **710** into a form processable by a trained model(s) using sentence embedding techniques as known in the art. Sentence embedding results in the ASR output data **710** including text in a structure that enables the trained models of the shortlister component **850** to operate on the ASR output data **710**. For example, an embedding of the ASR output data **710** may be a vector representation of the ASR output data **710**.

(124) The shortlister component **850** may make binary determinations (e.g., yes or no) regarding which domains relate to the ASR output data **710**. The shortlister component **850** may make such determinations using the one or more trained models described herein above. If the shortlister component **850** implements a single trained model for each domain, the shortlister component **850** may simply run the models that are associated with enabled domains as indicated in a user profile associated with the device **110** and/or user that originated the user input.

(125) The shortlister component **850** may generate n-best list data representing domains that may execute with respect to the user input represented in the ASR output data **710**. The size of the n-best list represented in the n-best list data is configurable. In an example, the n-best list data may indicate every domain of the system as well as contain an indication, for each domain, regarding whether the domain is likely capable to execute the user input represented in the ASR output data **710**. In another example, instead of indicating every domain of the system, the n-best list data may only indicate the domains that are likely to be able to execute the user input represented in the ASR output data **710**. In yet another example, the shortlister component **850** may implement thresholding such that the n-best list data may indicate no more than a maximum number of domains that may execute the user input represented in the ASR output data **710**. In an example, the threshold number of domains that may be represented in the n-best list data is ten. In another example, the domains included in the n-best list data may be limited by a threshold a score, where only domains indicating a likelihood to handle the user input is above a certain score (as determined by processing the ASR output data **710** by the shortlister component **850** relative to such domains) are included in the n-best list data.

(126) The ASR output data **710** may correspond to more than one ASR hypothesis. When this occurs, the shortlister component **850** may output a different n-best list (represented in the n-best list data) for each ASR hypothesis. Alternatively, the shortlister component **850** may output a single n-best list representing the domains that are related to the multiple ASR hypotheses represented in the ASR output data **710**.

(127) As indicated above, the shortlister component **850** may implement thresholding such that an n-best list output therefrom may include no more than a threshold number of entries. If the ASR output data **710** includes more than one ASR hypothesis, the n-best list output by the shortlister component **850** may include no more than a threshold number of entries irrespective of the number of ASR hypotheses output by the ASR component **650**. Alternatively or in addition, the n-best list output by the shortlister component **850** may include no more than a threshold number of entries for each ASR hypothesis (e.g., no more than five entries for a first ASR hypothesis, no more than

five entries for a second ASR hypothesis, etc.).

(128) In addition to making a binary determination regarding whether a domain potentially relates to the ASR output data **710**, the shortlister component **850** may generate confidence scores representing likelihoods that domains relate to the ASR output data **710**. If the shortlister component **850** implements a different trained model for each domain, the shortlister component **850** may generate a different confidence score for each individual domain trained model that is run. If the shortlister component **850** runs the models of every domain when ASR output data **710** is received, the shortlister component **850** may generate a different confidence score for each domain of the system. If the shortlister component **850** runs the models of only the domains that are associated with skills indicated as enabled in a user profile associated with the device **110** and/or user that originated the user input, the shortlister component **850** may only generate a different confidence score for each domain associated with at least one enabled skill. If the shortlister component **850** implements a single trained model with domain specifically trained portions, the shortlister component **850** may generate a different confidence score for each domain who's specifically trained portion is run. The shortlister component **850** may perform matrix vector modification to obtain confidence scores for all domains of the system in a single instance of processing of the ASR output data **710**.

(129) FIG. **9** is a conceptual diagram of text-to-speech components according to embodiments of the present disclosure. The TTS component **680** may process the text data to determine semantic meaning using one or more models as described below. The models may be updated by the techniques described herein. The device(s) **110** and/or system(s) **120** may include the TTS component **680**. The TTS component **680** may located across different physical and/or virtual machines. Components of a system that may be used to perform unit selection, parametric TTS processing, and/or model-based audio synthesis are shown in FIG. **9**. As shown in FIG. **9**, the TTS component/processor **680** may include a TTS front end **916**, a speech synthesis engine **918**, TTS unit storage **972**, TTS parametric storage **980**, and a TTS back end **934**. The TTS unit storage **972** may include, among other things, voice inventories **978***a*-**978***n* that may include pre-recorded audio segments (called units) to be used by the unit selection engine **930** when performing unit selection synthesis as described below. The TTS parametric storage **980** may include, among other things, parametric settings **968***a*-**968***n* that may be used by the parametric synthesis engine **932** when performing parametric synthesis as described below. A particular set of parametric settings **968** may correspond to a particular voice profile (e.g., whispered speech, excited speech, etc.).

(130) In various embodiments of the present disclosure, model-based synthesis of audio data may be performed using by a speech model **922** and a TTS front end **916**. The TTS front end **916** may be the same as front ends used in traditional unit selection or parametric systems. In other embodiments, some or all of the components of the TTS front end **916** are based on other trained models. The present disclosure is not, however, limited to any particular type of TTS front end **916**. The speech model **922** may be used to synthesize speech without requiring the TTS unit storage **972** or the TTS parametric storage **980**, as described in greater detail below.

(131) TTS component receives text data **910**. Although the text data **910** in FIG. **9** is input into the TTS component **680**, it may be output by other component(s) (such as a skill **690**, NLU component **660**, NLG component **679** or other component) and may be intended for output by the system. Thus in certain instances text data **910** may be referred to as “output text data.” Further, the data **910** may not necessarily be text, but may include other data (such as symbols, code, other data, etc.) that may reference text (such as an indicator of a word) that is to be synthesized. Thus data **910** may come in a variety of forms. The TTS front end **916** transforms the data **910** (from, for example, an application, user, device, or other data source) into a symbolic linguistic representation, which may include linguistic context features such as phoneme data, punctuation data, syllable-level features, word-level features, and/or emotion, speaker, accent, or other features for processing by the speech synthesis engine **918**. The syllable-level features may include syllable emphasis, syllable speech

rate, syllable inflection, or other such syllable-level features; the word-level features may include word emphasis, word speech rate, word inflection, or other such word-level features. The emotion features may include data corresponding to an emotion associated with the text data **910**, such as surprise, anger, or fear. The speaker features may include data corresponding to a type of speaker, such as sex, age, or profession. The accent features may include data corresponding to an accent associated with the speaker, such as Southern, Boston, English, French, or other such accent.

(132) The TTS front end **916** may also process other input data **915**, such as text tags or text metadata, that may indicate, for example, how specific words should be pronounced, for example by indicating the desired output speech quality in tags formatted according to the speech synthesis markup language (SSML) or in some other form. For example, a first text tag may be included with text marking the beginning of when text should be whispered (e.g., <begin whisper>) and a second tag may be included with text marking the end of when text should be whispered (e.g., <end whisper>). The tags may be included in the text data **910** and/or the text for a TTS request may be accompanied by separate metadata indicating what text should be whispered (or have some other indicated audio characteristic). The speech synthesis engine **918** may compare the annotated phonetic units models and information stored in the TTS unit storage **972** and/or TTS parametric storage **980** for converting the input text into speech. The TTS front end **916** and speech synthesis engine **918** may include their own controller(s)/processor(s) and memory or they may use the controller/processor and memory of the system **120**, device **110**, or other device, for example. Similarly, the instructions for operating the TTS front end **916** and speech synthesis engine **918** may be located within the TTS component **680**, within the memory and/or storage of the system **120**, device **110**, or within an external device.

(133) Text data **910** input into the TTS component **680** may be sent to the TTS front end **916** for processing. The front end **916** may include components for performing text normalization, linguistic analysis, linguistic prosody generation, or other such components. During text normalization, the TTS front end **916** may first process the text input and generate standard text, converting such things as numbers, abbreviations (such as Apt., St., etc.), symbols ($, %, etc.) into the equivalent of written out words.

(134) During linguistic analysis, the TTS front end **916** may analyze the language in the normalized text to generate a sequence of phonetic units corresponding to the input text. This process may be referred to as grapheme-to-phoneme conversion. Phonetic units include symbolic representations of sound units to be eventually combined and output by the system as speech. Various sound units may be used for dividing text for purposes of speech synthesis. The TTS component **680** may process speech based on phonemes (individual sounds), half-phonemes, di-phones (the last half of one phoneme coupled with the first half of the adjacent phoneme), bi-phones (two consecutive phonemes), syllables, words, phrases, sentences, or other units. Each word may be mapped to one or more phonetic units. Such mapping may be performed using a language dictionary stored by the system, for example in the TTS unit storage **972**. The linguistic analysis performed by the TTS front end **916** may also identify different grammatical components such as prefixes, suffixes, phrases, punctuation, syntactic boundaries, or the like. Such grammatical components may be used by the TTS component **680** to craft a natural-sounding audio waveform output. The language dictionary may also include letter-to-sound rules and other tools that may be used to pronounce previously unidentified words or letter combinations that may be encountered by the TTS component **680**. Generally, the more information included in the language dictionary, the higher quality the speech output.

(135) Based on the linguistic analysis the TTS front end **916** may then perform linguistic prosody generation where the phonetic units are annotated with desired prosodic characteristics, also called acoustic features, which indicate how the desired phonetic units are to be pronounced in the eventual output speech. During this stage the TTS front end **916** may consider and incorporate any prosodic annotations that accompanied the text input to the TTS component **680**. Such acoustic

features may include syllable-level features, word-level features, emotion, speaker, accent, language, pitch, energy, duration, and the like. Application of acoustic features may be based on prosodic models available to the TTS component **680**. Such prosodic models indicate how specific phonetic units are to be pronounced in certain circumstances. A prosodic model may consider, for example, a phoneme's position in a syllable, a syllable's position in a word, a word's position in a sentence or phrase, neighboring phonetic units, etc. As with the language dictionary, a prosodic model with more information may result in higher quality speech output than prosodic models with less information. Further, a prosodic model and/or phonetic units may be used to indicate particular speech qualities of the speech to be synthesized, where those speech qualities may match the speech qualities of input speech (for example, the phonetic units may indicate prosodic characteristics to make the ultimately synthesized speech sound like a whisper based on the input speech being whispered).

(136) The output of the TTS front end **916**, which may be referred to as a symbolic linguistic representation, may include a sequence of phonetic units annotated with prosodic characteristics. This symbolic linguistic representation may be sent to the speech synthesis engine **918**, which may also be known as a synthesizer, for conversion into an audio waveform of speech for output to an audio output device and eventually to a user. The speech synthesis engine **918** may be configured to convert the input text into high-quality natural-sounding speech in an efficient manner. Such high-quality speech may be configured to sound as much like a human speaker as possible, or may be configured to be understandable to a listener without attempts to mimic a precise human voice.

(137) The speech synthesis engine **918** may perform speech synthesis using one or more different methods. In one method of synthesis called unit selection, described further below, a unit selection engine **930** matches the symbolic linguistic representation created by the TTS front end **916** against a database of recorded speech, such as a database (e.g., TTS unit storage **972**) storing information regarding one or more voice corpuses (e.g., voice inventories **978***a-n*). Each voice inventory may correspond to various segments of audio that was recorded by a speaking human, such as a voice actor, where the segments are stored in an individual inventory **978** as acoustic units (e.g., phonemes, diphones, etc.). Each stored unit of audio may also be associated with an index listing various acoustic properties or other descriptive information about the unit. Each unit includes an audio waveform corresponding with a phonetic unit, such as a short .wav file of the specific sound, along with a description of various features associated with the audio waveform. For example, an index entry for a particular unit may include information such as a particular unit's pitch, energy, duration, harmonics, center frequency, where the phonetic unit appears in a word, sentence, or phrase, the neighboring phonetic units, or the like. The unit selection engine **930** may then use the information about each unit to select units to be joined together to form the speech output.

(138) The unit selection engine **930** matches the symbolic linguistic representation against information about the spoken audio units in the database. The unit database may include multiple examples of phonetic units to provide the system with many different options for concatenating units into speech. Matching units which are determined to have the desired acoustic qualities to create the desired output audio are selected and concatenated together (for example by a synthesis component **920**) to form output audio data **990** representing synthesized speech. Using all the information in the unit database, a unit selection engine **930** may match units to the input text to select units that can form a natural sounding waveform. One benefit of unit selection is that, depending on the size of the database, a natural sounding speech output may be generated. As described above, the larger the unit database of the voice corpus, the more likely the system will be able to construct natural sounding speech.

(139) In another method of synthesis—called parametric synthesis—parameters such as frequency, volume, noise, are varied by a parametric synthesis engine **932**, digital signal processor or other audio generation device to create an artificial speech waveform output. Parametric synthesis uses a computerized voice generator, sometimes called a vocoder. Parametric synthesis may use an

acoustic model and various statistical techniques to match a symbolic linguistic representation with desired output speech parameters. Using parametric synthesis, a computing system (for example, a synthesis component **920**) can generate audio waveforms having the desired acoustic properties. Parametric synthesis may include the ability to be accurate at high processing speeds, as well as the ability to process speech without large databases associated with unit selection, but also may produce an output speech quality that may not match that of unit selection. Unit selection and parametric techniques may be performed individually or combined together and/or combined with other synthesis techniques to produce speech audio output.

(140) The TTS component **680** may be configured to perform TTS processing in multiple languages. For each language, the TTS component **680** may include specially configured data, instructions and/or components to synthesize speech in the desired language(s). To improve performance, the TTS component **680** may revise/update the contents of the TTS unit storage **972** based on feedback of the results of TTS processing, thus enabling the TTS component **680** to improve speech synthesis.

(141) The TTS unit storage **972** may be customized for an individual user based on his/her individualized desired speech output. In particular, the speech unit stored in a unit database may be taken from input audio data of the user speaking. For example, to create the customized speech output of the system, the system may be configured with multiple voice inventories **978***a*-**978***n*, where each unit database is configured with a different "voice" to match desired speech qualities. Such voice inventories may also be linked to user accounts. The voice selected by the TTS component **680** may be used to synthesize the speech. For example, one voice corpus may be stored to be used to synthesize whispered speech (or speech approximating whispered speech), another may be stored to be used to synthesize excited speech (or speech approximating excited speech), and so on. To create the different voice corpuses a multitude of TTS training utterances may be spoken by an individual (such as a voice actor) and recorded by the system. The audio associated with the TTS training utterances may then be split into small audio segments and stored as part of a voice corpus. The individual speaking the TTS training utterances may speak in different voice qualities to create the customized voice corpuses, for example the individual may whisper the training utterances, say them in an excited voice, and so on. Thus the audio of each customized voice corpus may match the respective desired speech quality. The customized voice inventory **978** may then be used during runtime to perform unit selection to synthesize speech having a speech quality corresponding to the input speech quality.

(142) Additionally, parametric synthesis may be used to synthesize speech with the desired speech quality. For parametric synthesis, parametric features may be configured that match the desired speech quality. If simulated excited speech was desired, parametric features may indicate an increased speech rate and/or pitch for the resulting speech. Many other examples are possible. The desired parametric features for particular speech qualities may be stored in a "voice" profile (e.g., parametric settings **968**) and used for speech synthesis when the specific speech quality is desired. Customized voices may be created based on multiple desired speech qualities combined (for either unit selection or parametric synthesis). For example, one voice may be "shouted" while another voice may be "shouted and emphasized." Many such combinations are possible.

(143) Unit selection speech synthesis may be performed as follows. Unit selection includes a two-step process. First a unit selection engine **930** determines what speech units to use and then it combines them so that the particular combined units match the desired phonemes and acoustic features and create the desired speech output. Units may be selected based on a cost function which represents how well particular units fit the speech segments to be synthesized. The cost function may represent a combination of different costs representing different aspects of how well a particular speech unit may work for a particular speech segment. For example, a target cost indicates how well an individual given speech unit matches the features of a desired speech output (e.g., pitch, prosody, etc.). A join cost represents how well a particular speech unit matches an

adjacent speech unit (e.g., a speech unit appearing directly before or directly after the particular speech unit) for purposes of concatenating the speech units together in the eventual synthesized speech. The overall cost function is a combination of target cost, join cost, and other costs that may be determined by the unit selection engine **930**. As part of unit selection, the unit selection engine **930** chooses the speech unit with the lowest overall combined cost. For example, a speech unit with a very low target cost may not necessarily be selected if its join cost is high.

(144) The system may be configured with one or more voice corpuses for unit selection. Each voice corpus may include a speech unit database. The speech unit database may be stored in TTS unit storage **972** or in another storage component. For example, different unit selection databases may be stored in TTS unit storage **972**. Each speech unit database (e.g., voice inventory) includes recorded speech utterances with the utterances' corresponding text aligned to the utterances. A speech unit database may include many hours of recorded speech (in the form of audio waveforms, feature vectors, or other formats), which may occupy a significant amount of storage. The unit samples in the speech unit database may be classified in a variety of ways including by phonetic unit (phoneme, diphone, word, etc.), linguistic prosodic label, acoustic feature sequence, speaker identity, etc. The sample utterances may be used to create mathematical models corresponding to desired audio output for particular speech units. When matching a symbolic linguistic representation the speech synthesis engine **918** may attempt to select a unit in the speech unit database that most closely matches the input text (including both phonetic units and prosodic annotations). Generally, the larger the voice corpus/speech unit database the better the speech synthesis may be achieved by virtue of the greater number of unit samples that may be selected to form the precise desired speech output.

(145) Vocoder-based parametric speech synthesis may be performed as follows. A TTS component **680** may include an acoustic model, or other models, which may convert a symbolic linguistic representation into a synthetic acoustic waveform of the text input based on audio signal manipulation. The acoustic model includes rules which may be used by the parametric synthesis engine **932** to assign specific audio waveform parameters to input phonetic units and/or prosodic annotations. The rules may be used to calculate a score representing a likelihood that a particular audio output parameter(s) (such as frequency, volume, etc.) corresponds to the portion of the input symbolic linguistic representation from the TTS front end **916**.

(146) The parametric synthesis engine **932** may use a number of techniques to match speech to be synthesized with input phonetic units and/or prosodic annotations. One common technique is using Hidden Markov Models (HMMs). HMMs may be used to determine probabilities that audio output should match textual input. HMMs may be used to translate from parameters from the linguistic and acoustic space to the parameters to be used by a vocoder (the digital voice encoder) to artificially synthesize the desired speech. Using HMMs, a number of states are presented, in which the states together represent one or more potential acoustic parameters to be output to the vocoder and each state is associated with a model, such as a Gaussian mixture model. Transitions between states may also have an associated probability, representing a likelihood that a current state may be reached from a previous state. Sounds to be output may be represented as paths between states of the HMM and multiple paths may represent multiple possible audio matches for the same input text. Each portion of text may be represented by multiple potential states corresponding to different known pronunciations of phonemes and their parts (such as the phoneme identity, stress, accent, position, etc.). An initial determination of a probability of a potential phoneme may be associated with one state. As new text is processed by the speech synthesis engine **918**, the state may change or stay the same, based on the processing of the new text. For example, the pronunciation of a previously processed word might change based on later processed words. A Viterbi algorithm may be used to find the most likely sequence of states based on the processed text. The HMMs may generate speech in parameterized form including parameters such as fundamental frequency (f0), noise envelope, spectral envelope, etc. that are translated by a vocoder into audio segments. The

output parameters may be configured for particular vocoders such as a STRAIGHT vocoder, TANDEM-STRAIGHT vocoder, WORLD vocoder, HNM (harmonic plus noise) based vocoders, CELP (code-excited linear prediction) vocoders, GlottHMM vocoders, HSM (harmonic/stochastic model) vocoders, or others.

(147) In addition to calculating potential states for one audio waveform as a potential match to a phonetic unit, the parametric synthesis engine **932** may also calculate potential states for other potential audio outputs (such as various ways of pronouncing a particular phoneme or diphone) as potential acoustic matches for the acoustic unit. In this manner multiple states and state transition probabilities may be calculated.

(148) The probable states and probable state transitions calculated by the parametric synthesis engine **932** may lead to a number of potential audio output sequences. Based on the acoustic model and other potential models, the potential audio output sequences may be scored according to a confidence level of the parametric synthesis engine **932**. The highest scoring audio output sequence, including a stream of parameters to be synthesized, may be chosen and digital signal processing may be performed by a vocoder or similar component to create an audio output including synthesized speech waveforms corresponding to the parameters of the highest scoring audio output sequence and, if the proper sequence was selected, also corresponding to the input text. The different parametric settings **968**, which may represent acoustic settings matching a particular parametric “voice”, may be used by the synthesis component **920** to ultimately create the output audio data **990**.

(149) When performing unit selection, after a unit is selected by the unit selection engine **930**, the audio data corresponding to the unit may be passed to the synthesis component **920**. The synthesis component **920** may then process the audio data of the unit to create modified audio data where the modified audio data reflects a desired audio quality. The synthesis component **920** may store a variety of operations that can convert unit audio data into modified audio data where different operations may be performed based on the desired audio effect (e.g., whispering, shouting, etc.).

(150) As an example, input text may be received along with metadata, such as SSML tags, indicating that a selected portion of the input text should be whispered when output by the TTS module **680**. For each unit that corresponds to the selected portion, the synthesis component **920** may process the audio data for that unit to create a modified unit audio data. The modified unit audio data may then be concatenated to form the output audio data **990**. The modified unit audio data may also be concatenated with non-modified audio data depending on when the desired whispered speech starts and/or ends. While the modified audio data may be sufficient to imbue the output audio data with the desired audio qualities, other factors may also impact the ultimate output of audio such as playback speed, background effects, or the like, that may be outside the control of the TTS module **680**. In that case, other output data **985** may be output along with the output audio data **990** so that an ultimate playback device (e.g., device **110**) receives instructions for playback that can assist in creating the desired output audio. Thus, the other output data **985** may include instructions or other data indicating playback device settings (such as volume, playback rate, etc.) or other data indicating how output audio data including synthesized speech should be output. For example, for whispered speech, the output audio data **990** may include other output data **985** that may include a prosody tag or other indicator that instructs the device **110** to slow down the playback of the output audio data **990**, thus making the ultimate audio sound more like whispered speech, which is typically slower than normal speech. In another example, the other output data **985** may include a volume tag that instructs the device **110** to output the speech at a volume level less than a current volume setting of the device **110**, thus improving the quiet whisper effect.

(151) FIG. **10** is a conceptual diagram of an image processing component **640** according to embodiments of the present disclosure. The image processing component **640** may process still and/or video image data using one or more models as described below. The models may be updated by the techniques described herein. The device(s) **110** and/or system(s) **120** may include the image

processing component **640**. The image processing component **640** may located across different physical and/or virtual machines. The image processing component **640** may receive and analyze image data (which may include single images or a plurality of images such as in a video feed). The image processing component **640** may work with other components of the system **120** to perform various operations. For example the image processing component **640** may assist with user recognition using image data. The image processing component **640** may also include or otherwise be associated with image data storage **1070** which may store aspects of image data used by image processing component **640**. The image data may be of different formats such as JPEG, GIF, BMP, MPEG, video formats, and the like.

(152) Image matching algorithms, such as those used by image processing component **640**, may take advantage of the fact that an image of an object or scene contains a number of feature points. Feature points are specific points in an image which are robust to changes in image rotation, scale, viewpoint, or lighting conditions. This means that these feature points will often be present in both the images to be compared, even if the two images differ. These feature points may also be known as “points of interest.” Therefore, a first stage of the image matching algorithm may include finding these feature points in the image. An image pyramid may be constructed to determine the feature points of an image. An image pyramid is a scale-space representation of the image, e.g., it contains various pyramid images, each of which is a representation of the image at a particular scale. The scale-space representation enables the image matching algorithm to match images that differ in overall scale (such as images taken at different distances from an object). Pyramid images may be smoothed and downsampled versions of an original image.

(153) To build a database of object images, with multiple objects per image, a number of different images of an object may be taken from different viewpoints. From those images, feature points may be extracted and pyramid images constructed. Multiple images from different points of view of each particular object may be taken and linked within the database (for example within a tree structure described below). The multiple images may correspond to different viewpoints of the object sufficient to identify the object from any later angle that may be included in a user's query image. For example, a shoe may look very different from a bottom view than from a top view than from a side view. For certain objects, this number of different image angles may be 6 (top, bottom, left side, right side, front, back), for other objects this may be more or less depending on various factors, including how many images should be taken to ensure the object may be recognized in an incoming query image. With different images of the object available, it is more likely that an incoming image from a user may be recognized by the system and the object identified, even if the user's incoming image is taken at a slightly different angle.

(154) This process may be repeated for multiple objects. For large databases, such as an online shopping database where a user may submit an image of an object to be identified, this process may be repeated thousands, if not millions of times to construct a database of images and data for image matching. The database also may continually be updated and/or refined to account for a changing catalog of objects to be recognized.

(155) When configuring the database, pyramid images, feature point data, and/or other information from the images or objects may be used to cluster features and build a tree of objects and images, where each node of the tree will keep lists of objects and corresponding features. The tree may be configured to group visually significant subsets of images/features to ease matching of submitted images for object detection. Data about objects to be recognized may be stored by the system in image data **1070**, profile storage **670**, or other storage component.

(156) Image selection component **1020** may select desired images from input image data to use for image processing at runtime. For example, input image data may come from a series of sequential images, such as a video stream where each image is a frame of the video stream. These incoming images need to be sorted to determine which images will be selected for further object recognition processing as performing image processing on low quality images may result in an undesired user

experience. To avoid such an undesirable user experience, the time to perform the complete recognition process, from first starting the video feed to delivering results to the user, should be as short as possible. As images in a video feed may come in rapid succession, the image processing component **640** may be configured to select or discard an image quickly so that the system can, in turn, quickly process the selected image and deliver results to a user. The image selection component **1020** may select an image for object recognition by computing a metric/feature for each frame in the video feed and selecting an image for processing if the metric exceeds a certain threshold. While FIG. **10** illustrates image selection component **1020** as part of system **120**, it may also be located on device **110** so that the device may select only desired image(s) to send to system **120**, thus avoiding sending too much image data to system **120** (thus expending unnecessary computing/communication resources). Thus, the device may select only the best quality images for purposes of image analysis.

(157) The metrics used to select an image may be general image quality metrics (focus, sharpness, motion, etc.) or may be customized image quality metrics. The metrics may be computed by software components or hardware components. For example, the metrics may be derived from output of device sensors such as a gyroscope, accelerometer, field sensors, inertial sensors, camera metadata, or other components. The metrics may thus be image based (such as a statistic derived from an image or taken from camera metadata like focal length or the like) or may be non-image based (for example, motion data derived from a gyroscope, accelerometer, GPS sensor, etc.). As images from the video feed are obtained by the system, the system, such as a device, may determine metric values for the image. One or more metrics may be determined for each image. To account for temporal fluctuation, the individual metrics for each respective image may be compared to the metric values for previous images in the image feed and thus a historical metric value for the image and the metric may be calculated. This historical metric may also be referred to as a historical metric value. The historical metric values may include representations of certain metric values for the image compared to the values for that metric for a group of different images in the same video feed. The historical metric(s) may be processed using a trained classifier model to select which images are suitable for later processing.

(158) For example, if a particular image is to be measured using a focus metric, which is a numerical representation of the focus of the image, the focus metric may also be computed for the previous N frames to the particular image. N is a configurable number and may vary depending on system constraints such as latency, accuracy, etc. For example, N may be 30 image frames, representing, for example, one second of video at a video feed of 30 frames-per-second. A mean of the focus metrics for the previous N images may be computed, along with a standard deviation for the focus metric. For example, for an image number X+1 in a video feed sequence, the previous N images, may have various metric values associated with each of them. Various metrics such as focus, motion, and contrast are discussed, but others are possible. A value for each metric for each of the N images may be calculated, and then from those individual values, a mean value and standard deviation value may be calculated. The mean and standard deviation (STD) may then be used to calculate a normalized historical metric value, for example STD(metric)/MEAN(metric). Thus, the value of a historical focus metric at a particular image may be the STD divided by the mean for the focus metric for the previous N frames. For example, historical metrics (HIST) for focus, motion, and contrast may be expressed as:

(159) $\mathrm{HIST}_{\mathrm{Focus}} = \frac{\mathrm{STD}_{\mathrm{Focus}}}{\mathrm{MEAN}_{\mathrm{Focus}}} \mathrm{HIST}_{\mathrm{Motion}} = \frac{\mathrm{STD}_{\mathrm{Motion}}}{\mathrm{MEAN}_{\mathrm{Motion}}} \mathrm{HIST}_{\mathrm{Contrast}} = \frac{\mathrm{STD}_{\mathrm{Contrast}}}{\mathrm{MEAN}_{\mathrm{Contrast}}}$

(160) In one embodiment the historical metric may be further normalized by dividing the above historical metrics by the number of frames N, particularly in situations where there are small number of frames under consideration for the particular time window. The historical metrics may be recalculated with each new image frame that is received as part of the video feed. Thus, each frame of an incoming video feed may have a different historical metric from the frame before. The metrics for a particular image of a video feed may be compared historical metrics to select a

desirable image on which to perform image processing.
(161) Image selection component **1020** may perform various operations to identify potential locations in an image that may contain recognizable text. This process may be referred to as glyph region detection. A glyph is a text character that has yet to be recognized. If a glyph region is detected, various metrics may be calculated to assist the eventual optical character recognition (OCR) process. For example, the same metrics used for overall image selection may be re-used or recalculated for the specific glyph region. Thus, while the entire image may be of sufficiently high quality, the quality of the specific glyph region (i.e., focus, contrast, intensity, etc.) may be measured. If the glyph region is of poor quality, the image may be rejected for purposes of text recognition.
(162) Image selection component **1020** may generate a bounding box that bounds a line of text. The bounding box may bound the glyph region. Value(s) for image/region suitability metric(s) may be calculated for the portion of the image in the bounding box. Value(s) for the same metric(s) may also be calculated for the portion of the image outside the bounding box. The value(s) for inside the bounding box may then be compared to the value(s) outside the bounding box to make another determination on the suitability of the image. This determination may also use a classifier.
(163) Additional features may be calculated for determining whether an image includes a text region of sufficient quality for further processing. The values of these features may also be processed using a classifier to determine whether the image contains true text character/glyphs or is otherwise suitable for recognition processing. To locally classify each candidate character location as a true text character/glyph location, a set of features that capture salient characteristics of the candidate location is extracted from the local pixel pattern. Such features may include aspect ratio (bounding box width/bounding box height), compactness (4*π*candidate glyph area/(perimeter).sup.2), solidity (candidate glyph area/bounding box area), stroke-width to width ratio (maximum stroke width/bounding box width), stroke-width to height ratio (maximum stroke width/bounding box height), convexity (convex hull perimeter/perimeter), raw compactness (4*π* (candidate glyph number of pixels)/(perimeter).sup.2), number of holes in candidate glyph, or other features. Other candidate region identification techniques may be used. For example, the system may use techniques involving maximally stable extremal regions (MSERs). Instead of MSERs (or in conjunction with MSERs), the candidate locations may be identified using histogram of oriented gradients (HoG) and Gabor features.
(164) If an image is sufficiently high quality, it may be selected by image selection **1020** for sending to another component (e.g., from device to system **120**) and/or for further processing, such as text recognition, object detection/resolution, etc.
(165) The feature data calculated by image selection component **1020** may be sent to other components such as text recognition component **1040**, objection detection component **1030**, object resolution component **1050**, etc. so that those components may use the feature data in their operations. Other preprocessing operations such as masking, binarization, etc. may be performed on image data prior to recognition/resolution operations. Those preprocessing operations may be performed by the device prior to sending image data or by system **120**.
(166) Object detection component **1030** may be configured to analyze image data to identify one or more objects represented in the image data. Various approaches can be used to attempt to recognize and identify objects, as well as to determine the types of those objects and applications or actions that correspond to those types of objects, as is known or used in the art. For example, various computer vision algorithms can be used to attempt to locate, recognize, and/or identify various types of objects in an image or video sequence. Computer vision algorithms can utilize various different approaches, as may include edge matching, edge detection, recognition by parts, gradient matching, histogram comparisons, interpretation trees, and the like.
(167) The object detection component **1030** may process at least a portion of the image data to determine feature data. The feature data is indicative of one or more features that are depicted in

the image data. For example, the features may be face data, or other objects, for example as represented by stored data in profile storage **670**. Other examples of features may include shapes of body parts or other such features that identify the presence of a human. Other examples of features may include edges of doors, shadows on the wall, texture on the walls, portions of artwork in the environment, and so forth to identify a space. The object detection component **1030** may compare detected features to stored data (e.g., in profile storage **670**, image data **1070**, or other storage) indicating how detected features may relate to known objects for purposes of object detection.
(168) Various techniques may be used to determine the presence of features in image data. For example, one or more of a Canny detector, Sobel detector, difference of Gaussians, features from accelerated segment test (FAST) detector, scale-invariant feature transform (SIFT), speeded up robust features (SURF), color SIFT, local binary patterns (LBP), trained convolutional neural network, or other detection methodologies may be used to determine features in the image data. A feature that has been detected may have an associated descriptor that characterizes that feature. The descriptor may comprise a vector value in some implementations. For example, the descriptor may comprise data indicative of the feature with respect to many (e.g., 256) different dimensions.
(169) One statistical algorithm that may be used for geometric matching of images is the Random Sample Consensus (RANSAC) algorithm, although other variants of RANSAC-like algorithms or other statistical algorithms may also be used. In RANSAC, a small set of putative correspondences is randomly sampled. Thereafter, a geometric transformation is generated using these sampled feature points. After generating the transformation, the putative correspondences that fit the model are determined. The putative correspondences that fit the model and are geometrically consistent and called “inliers.” The inliers are pairs of feature points, one from each image, that may correspond to each other, where the pair fits the model within a certain comparison threshold for the visual (and other) contents of the feature points, and are geometrically consistent (as explained below relative to motion estimation). A total number of inliers may be determined. The above mentioned steps may be repeated until the number of repetitions/trials is greater than a predefined threshold or the number of inliers for the image is sufficiently high to determine an image as a match (for example the number of inliers exceeds a threshold). The RANSAC algorithm returns the model with the highest number of inliers corresponding to the model.
(170) To further test pairs of putative corresponding feature points between images, after the putative correspondences are determined, a topological equivalence test may be performed on a subset of putative correspondences to avoid forming a physically invalid transformation. After the transformation is determined, an orientation consistency test may be performed. An offset point may be determined for the feature points in the subset of putative correspondences in one of the images. Each offset point is displaced from its corresponding feature point in the direction of the orientation of that feature point. The transformation is discarded based on orientation of the feature points obtained from the feature points in the subset of putative correspondences if any one of the images being matched and its offset point differs from an estimated orientation by a predefined limit. Subsequently, motion estimation may be performed using the subset of putative correspondences which satisfy the topological equivalence test.
(171) Motion estimation (also called geometric verification) may determine the relative differences in position between corresponding pairs of putative corresponding feature points. A geometric relationship between putative corresponding feature points may determine where in one image (e.g., the image input to be matched) a particular point is found relative to that potentially same point in the putatively matching image (i.e., a database image). The geometric relationship between many putatively corresponding feature point pairs may also be determined, thus creating a potential map between putatively corresponding feature points across images. Then the geometric relationship of these points may be compared to determine if a sufficient number of points correspond (that is, if the geometric relationship between point pairs is within a certain threshold score for the geometric relationship), thus indicating that one image may represent the same real-

world physical object, albeit from a different point of view. Thus, the motion estimation may determine that the object in one image is the same as the object in another image, only rotated by a certain angle or viewed from a different distance, etc.
(172) The above processes of image comparing feature points and performing motion estimation across putative matching images may be performed multiple times for a particular query image to compare the query image to multiple potential matches among the stored database images. Dozens of comparisons may be performed before one (or more) satisfactory matches that exceed the relevant thresholds (for both matching feature points and motion estimation) may be found. The thresholds may also include a confidence threshold, which compares each potential matching image with a confidence score that may be based on the above processing. If the confidence score exceeds a certain high threshold, the system may stop processing additional candidate matches and simply select the high confidence match as the final match. Or if, the confidence score of an image is within a certain range, the system may keep the candidate image as a potential match while continuing to search other database images for potential matches. In certain situations, multiple database images may exceed the various matching/confidence thresholds and may be determined to be candidate matches. In this situation, a comparison of a weight or confidence score may be used to select the final match, or some combination of candidate matches may be used to return results. The system may continue attempting to match an image until a certain number of potential matches are identified, a certain confidence score is reached (either individually with a single potential match or among multiple matches), or some other search stop indicator is triggered. For example, a weight may be given to each object of a potential matching database image. That weight may incrementally increase if multiple query images (for example, multiple frames from the same image stream) are found to be matches with database images of a same object. If that weight exceeds a threshold, a search stop indicator may be triggered and the corresponding object selected as the match.
(173) Once an object is detected by object detection component **1030** the system may determine which object is actually seen using object resolution component **1050**. Thus, one component, such as object detection component **1030**, may detect if an object is represented in an image while another component, object resolution component **1050** may determine which object is actually represented. Although illustrated as separate components, the system may also be configured so that a single component may perform both object detection and object resolution.
(174) For example, when a database image is selected as a match to the query image, the object in the query image may be determined to be the object in the matching database image. An object identifier associated with the database image (such as a product ID or other identifier) may be used to return results to a user, along the lines of “I see you holding object X” along with other information, such giving the user information about the object. If multiple potential matches are returned (such as when the system can't determine exactly what object is found or if multiple objects appear in the query image) the system may indicate to the user that multiple potential matching objects are found and may return information/options related to the multiple objects.
(175) In another example, object detection component **1030** may determine that a type of object is represented in image data and object resolution component **1050** may then determine which specific object is represented. The object resolution component **1050** may also make available specific data about a recognized object to further components so that further operations may be performed with regard to the resolved object.
(176) Object detection component **1030** may be configured to process image data to detect a representation of an approximately two-dimensional (2D) object (such as a piece of paper) or a three-dimensional (3D) object (such as a face). Such recognition may be based on available stored data (e.g., **670**, **1070**, etc.) which in turn may have been provided through an image data ingestion process managed by image data ingestion component **1010**. Various techniques may be used to determine the presence of features in image data. For example, one or more of a Canny detector,

Sobel detector, difference of Gaussians, features from accelerated segment test (FAST) detector, scale-invariant feature transform (SIFT), speeded up robust features (SURF), color SIFT, local binary patterns (LBP), trained convolutional neural network, or other detection methodologies may be used to determine features in the image data. A feature that has been detected may have an associated descriptor that characterizes that feature. The descriptor may comprise a vector value in some implementations. For example, the descriptor may comprise data indicative of the feature with respect to many (e.g., 256) different dimensions.

(177) Various machine learning techniques may be used to train and operate models to perform various steps described herein, such as user recognition, sentiment detection, image processing, dialog management, etc. Models may be trained and operated according to various machine learning techniques. Such techniques may include, for example, neural networks (such as deep neural networks and/or recurrent neural networks), inference engines, trained classifiers, etc. Examples of trained classifiers include Support Vector Machines (SVMs), neural networks, decision trees, AdaBoost (short for "Adaptive Boosting") combined with decision trees, and random forests. Focusing on SVM as an example, SVM is a supervised learning model with associated learning algorithms that analyze data and recognize patterns in the data, and which are commonly used for classification and regression analysis. Given a set of training examples, each marked as belonging to one of two categories, an SVM training algorithm builds a model that assigns new examples into one category or the other, making it a non-probabilistic binary linear classifier. More complex SVM models may be built with the training set identifying more than two categories, with the SVM determining which category is most similar to input data. An SVM model may be mapped so that the examples of the separate categories are divided by clear gaps. New examples are then mapped into that same space and predicted to belong to a category based on which side of the gaps they fall on. Classifiers may issue a "score" indicating which category the data most closely matches. The score may provide an indication of how closely the data matches the category.

(178) In order to apply the machine learning techniques, the machine learning processes themselves need to be trained. Training a machine learning component such as, in this case, one of the first or second models, requires establishing a "ground truth" for the training examples. In machine learning, the term "ground truth" refers to the accuracy of a training set's classification for supervised learning techniques. Various techniques may be used to train the models including backpropagation, statistical learning, supervised learning, semi-supervised learning, stochastic learning, or other known techniques.

(179) FIG. **11** is a block diagram conceptually illustrating a device **110** that may be used with the system. FIG. **12** is a block diagram conceptually illustrating example components of a remote device, such as the system **120**, which may assist with ASR processing, NLU processing, etc., and a skill system **625**. A system (**120**/**125**) may include one or more servers. A "server" as used herein may refer to a traditional server as understood in a server/client computing structure but may also refer to a number of different computing components that may assist with the operations discussed herein. For example, a server may include one or more physical computing components (such as a rack server) that are connected to other devices/components either physically and/or over a network and is capable of performing computing operations. A server may also include one or more virtual machines that emulates a computer system and is run on one or across multiple devices. A server may also include other combinations of hardware, software, firmware, or the like to perform operations discussed herein. The server(s) may be configured to operate using one or more of a client-server model, a computer bureau model, grid computing techniques, fog computing techniques, mainframe techniques, utility computing techniques, a peer-to-peer model, sandbox techniques, or other computing techniques.

(180) While the device **110** may operate locally to a user (e.g., within a same environment so the device may receive inputs and playback outputs for the user) he server/system **120** may be located

remotely from the device **110** as its operations may not require proximity to the user. The server/system **120** may be located in an entirely different location from the device **110** (for example, as part of a cloud computing system or the like) or may be located in a same environment as the device **110** but physically separated therefrom (for example a home server or similar device that resides in a user's home or business but perhaps in a closet, basement, attic, or the like). One benefit to the server/system **120** being in a user's home/business is that data used to process a command/return a response may be kept within the user's home, thus reducing potential privacy concerns.

(181) Multiple systems (**120**/**125**) may be included in the overall system **100** of the present disclosure, such as one or more systems **120** for performing ASR processing, one or more systems **120** for performing NLU processing, one or more skill systems **625**, etc. In operation, each of these systems may include computer-readable and computer-executable instructions that reside on the respective device (**120**/**125**), as will be discussed further below.

(182) Each of these devices (**110**/**120**/**125**) may include one or more controllers/processors (**1104**/**1204**), which may each include a central processing unit (CPU) for processing data and computer-readable instructions, and a memory (**1106**/**1206**) for storing data and instructions of the respective device. The memories (**1106**/**1206**) may individually include volatile random access memory (RAM), non-volatile read only memory (ROM), non-volatile magnetoresistive memory (MRAM), and/or other types of memory. Each device (**110**/**120**/**125**) may also include a data storage component (**1108**/**1208**) for storing data and controller/processor-executable instructions. Each data storage component (**1108**/**1208**) may individually include one or more non-volatile storage types such as magnetic storage, optical storage, solid-state storage, etc. Each device (**110**/**120**/**125**) may also be connected to removable or external non-volatile memory and/or storage (such as a removable memory card, memory key drive, networked storage, etc.) through respective input/output device interfaces (**1102**/**1202**).

(183) Computer instructions for operating each device (**110**/**120**/**125**) and its various components may be executed by the respective device's controller(s)/processor(s) (**1104**/**1204**), using the memory (**1106**/**1206**) as temporary “working” storage at runtime. A device's computer instructions may be stored in a non-transitory manner in non-volatile memory (**1106**/**1206**), storage (**1108**/**1208**), or an external device(s). Alternatively, some or all of the executable instructions may be embedded in hardware or firmware on the respective device in addition to or instead of software.

(184) Each device (**110**/**120**/**125**) includes input/output device interfaces (**1102**/**1202**). A variety of components may be connected through the input/output device interfaces (**1102**/**1202**), as will be discussed further below. Additionally, each device (**110**/**120**/**125**) may include an address/data bus (**1124**/**1224**) for conveying data among components of the respective device. Each component within a device (**110**/**120**/**125**) may also be directly connected to other components in addition to (or instead of) being connected to other components across the bus (**1124**/**1224**).

(185) Referring to FIG. **11**, the device **110** may include input/output device interfaces **1102** that connect to a variety of components such as an audio output component such as a speaker **1112**, a wired headset or a wireless headset (not illustrated), or other component capable of outputting audio. The device **110** may also include an audio capture component. The audio capture component may be, for example, a microphone **1120** or array of microphones, a wired headset or a wireless headset (not illustrated), etc. If an array of microphones is included, approximate distance to a sound's point of origin may be determined by acoustic localization based on time and amplitude differences between sounds captured by different microphones of the array. The device **110** may additionally include a display **1116** for displaying content. The device **110** may further include a camera **1118**.

(186) Via antenna(s) **1122**, the input/output device interfaces **1102** may connect to one or more networks **199** via a wireless local area network (WLAN) (such as Wi-Fi) radio, Bluetooth, and/or

wireless network radio, such as a radio capable of communication with a wireless communication network such as a Long Term Evolution (LTE) network, WiMAX network, 3G network, 4G network, 5G network, etc. A wired connection such as Ethernet may also be supported. Through the network(s) **199**, the system may be distributed across a networked environment. The I/O device interface (**1102**/**1202**) may also include communication components that allow data to be exchanged between devices such as different physical servers in a collection of servers or other components.

(187) The components of the device(s) **110**, the system **120**, or a skill system **625** may include their own dedicated processors, memory, and/or storage. Alternatively, one or more of the components of the device(s) **110**, the system **120**, or a skill system **625** may utilize the I/O interfaces (**1102**/**1202**), processor(s) (**1104**/**1204**), memory (**1106**/**1206**), and/or storage (**1108**/**1208**) of the device(s) **110**, system **120**, or the skill system **625**, respectively. Thus, the ASR component **650** may have its own I/O interface(s), processor(s), memory, and/or storage; the NLU component **660** may have its own I/O interface(s), processor(s), memory, and/or storage; and so forth for the various components discussed herein.

(188) As noted above, multiple devices may be employed in a single system. In such a multi-device system, each of the devices may include different components for performing different aspects of the system's processing. The multiple devices may include overlapping components. The components of the device **110**, the system **120**, and a skill system **625**, as described herein, are illustrative, and may be located as a stand-alone device or may be included, in whole or in part, as a component of a larger device or system. As can be appreciated, a number of components may exist either on a system **120** and/or on device **110**. For example, language processing **692** (which may include ASR **650**), language output **693** (which may include NLG **679** and TTS **680**), etc., for example as illustrated in FIG. **6**. Unless expressly noted otherwise, the system version of such components may operate similarly to the device version of such components and thus the description of one version (e.g., the system version or the local version) applies to the description of the other version (e.g., the local version or system version) and vice-versa.

(189) As illustrated in FIG. **13**, multiple devices (**110***a*-**110***n*, **120**, **125**) may contain components of the system and the devices may be connected over a network(s) **199**. The network(s) **199** may include a local or private network or may include a wide network such as the Internet. Devices may be connected to the network(s) **199** through either wired or wireless connections. For example, a speech-detection device **110***a*, a smart phone **110***b*, a smart watch **110***c*, a tablet computer **110***d*, a vehicle **110***e*, a speech-detection device with display **110***f*, a display/smart television **110***g*, a washer/dryer **110***h*, a refrigerator **110***i*, a microwave **110***j*, etc. (e.g., a device such as a FireTV stick, Echo Auto or the like) may be connected to the network(s) **199** through a wireless service provider, over a Wi-Fi or cellular network connection, or the like. Other devices are included as network-connected support devices, such as the system **120**, the skill system(s) **625**, and/or others. The support devices may connect to the network(s) **199** through a wired connection or wireless connection. Networked devices may capture audio using one-or-more built-in or connected microphones or other audio capture devices, with processing performed by ASR components, NLU components, or other components of the same device or another device connected via the network(s) **199**, such as the ASR component **650**, the NLU component **660**, etc. of the system **120**.

(190) The concepts disclosed herein may be applied within a number of different devices and computer systems, including, for example, general-purpose computing systems, speech processing systems, and distributed computing environments.

(191) The above aspects of the present disclosure are meant to be illustrative. They were chosen to explain the principles and application of the disclosure and are not intended to be exhaustive or to limit the disclosure. Many modifications and variations of the disclosed aspects may be apparent to those of skill in the art. Persons having ordinary skill in the field of computers and speech processing should recognize that components and process steps described herein may be

interchangeable with other components or steps, or combinations of components or steps, and still achieve the benefits and advantages of the present disclosure. Moreover, it should be apparent to one skilled in the art, that the disclosure may be practiced without some or all of the specific details and steps disclosed herein. Further, unless expressly stated to the contrary, features/operations/components, etc. from one embodiment discussed herein may be combined with features/operations/components, etc. from another embodiment discussed herein.

(192) Aspects of the disclosed system may be implemented as a computer method or as an article of manufacture such as a memory device or non-transitory computer readable storage medium. The computer readable storage medium may be readable by a computer and may comprise instructions for causing a computer or other device to perform processes described in the present disclosure. The computer readable storage medium may be implemented by a volatile computer memory, non-volatile computer memory, hard drive, solid-state memory, flash drive, removable disk, and/or other media. In addition, components of system may be implemented as in firmware or hardware.

(193) Conditional language used herein, such as, among others, “can,” “could,” “might,” “may,” “e.g.,” and the like, unless specifically stated otherwise, or otherwise understood within the context as used, is generally intended to convey that certain embodiments include, while other embodiments do not include, certain features, elements and/or steps. Thus, such conditional language is not generally intended to imply that features, elements, and/or steps are in any way required for one or more embodiments or that one or more embodiments necessarily include logic for deciding, with or without other input or prompting, whether these features, elements, and/or steps are included or are to be performed in any particular embodiment. The terms “comprising,” “including,” “having,” and the like are synonymous and are used inclusively, in an open-ended fashion, and do not exclude additional elements, features, acts, operations, and so forth. Also, the term “or” is used in its inclusive sense (and not in its exclusive sense) so that when used, for example, to connect a list of elements, the term “or” means one, some, or all of the elements in the list.

(194) Disjunctive language such as the phrase “at least one of X, Y, Z,” unless specifically stated otherwise, is understood with the context as used in general to present that an item, term, etc., may be either X, Y, or Z, or any combination thereof (e.g., X, Y, and/or Z). Thus, such disjunctive language is not generally intended to, and should not, imply that certain embodiments require at least one of X, at least one of Y, or at least one of Z to each be present.

(195) As used in this disclosure, the term “a” or “one” may include one or more items unless specifically stated otherwise. Further, the phrase “based on” is intended to mean “based at least in part on” unless specifically stated otherwise.

Claims

1. A computer-implemented method comprising: processing, by a first device, first user input data using a first neural network model stored on the first device to determine output data responsive to the first user input data, the first neural network model including a plurality of layers; determining, using the output data, first parameters of a second neural network model, the second neural network model representing a version of the first neural network model updated based at least in part on the output data; receiving, from a second device, data representing a first protocol for determining first model update data to the second device, the first protocol specifying a subset of model parameters, representing fewer than all model parameters, to include in a model update; determining, using the first protocol and the second neural network model, a first subset of parameters corresponding to at least a first layer and a second layer of the plurality of layers; transforming the first subset using one or more operations indicated by the first protocol to generate the first model update data; sending the first model update data to the second device; sending a first indication of the first protocol to the second device, the first indication causing the second device to

determine the first subset of parameters based on the first model update data; receiving, from the second device, a second indication of a second protocol for receiving second model update data from the second device; receiving, from the second device, the second model update data, the second model update data based on an aggregation of the first model update data and at least third model update data received by the second device from at least a third device; and generating, by the first device using the second model update data and the second neural network model, a third neural network model representing a version of the second neural network model updated based on second model update data.

2. The computer-implemented method of claim 1, wherein: transforming the first subset includes reordering the at least first layer and second layer such that an order of the at least first layer and second layer in the first model update data is different from an order of the at least first layer and second layer in the plurality of layers.

3. The computer-implemented method of claim 1, wherein: the first neural network model additionally includes at least a third layer; and determining the first subset includes determining to omit, from the first model update data, parameters corresponding to the third layer.

4. The computer-implemented method of claim 1, wherein: transforming the first subset includes performing at least one mathematical function described by the first protocol on the parameters of the first subset to generate the first model update data; and performing the at least one mathematical function includes generating one or more new parameter values of the first subset.

5. A computer-implemented method comprising: processing, by a first device, first input data using a first machine learning model stored on the first device to determine first output data; determining, using the first output data, first parameters of a second machine learning model, the second machine learning model representing a version of the first machine learning model updated based at least in part on the first output data; determining a first protocol for generating first model update data to be sent to a second device, the protocol specifying a subset of model parameters, representing fewer than all model parameters, to include in a model update; determining, using the first protocol, a first subset of parameters of the second machine learning model; generating the first model update data using the first subset of parameters and one or more operations indicated by the first protocol; sending, to the second device, the first model update data; receiving, from the second device in response to the first model update data, second model update data; determining, by the first device using the second model update data, a second subset of parameters; and generating, by the first device using the second subset and the second machine learning model, a third machine learning model representing a version of the second machine learning model updated based at least in part on the second model update data.

6. The computer-implemented method of claim 5, further comprising: determining a dataset for validating the second model update data; processing the dataset using the second machine learning model to determine a first error rate; processing the dataset using the third machine learning model to determine a second error rate; and determining, based at least in part on the first error rate and the second error rate, to process subsequent input data using the third machine learning model.

7. The computer-implemented method of claim 5, further comprising: determining a dataset for validating the second model update data; processing the dataset using the second machine learning model to determine second output data; processing the dataset using the third machine learning model to determine third output data; and determining, based at least in part on the second output data and the third output data, to perform a partial update of the second machine learning model, wherein generating the third machine learning model includes performing a weighted operation using the first subset of parameters and the second subset of parameters.

8. The computer-implemented method of claim 5, wherein the second machine learning model includes at least an input layer for receiving input data, an output layer for outputting the first output data, and at least one internal layer configured to receive first data from the input layer and send second data to the output layer, the method further comprising: determining, using the first

protocol, to include a representation of the at least one internal layer in the first model update data; and determining the first subset of parameters based at least in part on first parameters corresponding to the at least one internal layer.
9. The computer-implemented method of claim 5, further comprising: receiving, from the second device in response to the first model update data, a second indication; and determining, based at least in part on the second indication, a second protocol for receiving the second model update data, wherein determining the second subset of parameters is further based at least in part on the second protocol.
10. The computer-implemented method of claim 5, wherein: the second machine learning model includes at least a first layer and a second layer; and generating the first model update data includes reordering the at least first layer and second layer such that an order of the at least first layer and second layer in the first model update data is different from an order of the at least first layer and second layer in the second machine learning model.
11. The computer-implemented method of claim 5, wherein: the second machine learning model includes at least a first layer and a second layer; and determining the first subset includes determining to include a representation of first parameters of the first layer in the first model update data, but not a representation of second parameters of the second layer.
12. The computer-implemented method of claim 5, wherein: generating the first model update data includes performing at least one mathematical function described by the first protocol on the parameters of the first subset; performing the at least one mathematical function includes generating one or more new parameter values; and the first subset includes the one or more new parameter values.
13. A system, comprising: at least one processor; and at least one memory comprising instructions that, when executed by the at least one processor, cause the system to: process, by a first device, first input data using a first machine learning model stored on the first device to determine first output data; determine, using the first output data, first parameters of a second machine learning model, the second machine learning model representing a version of the first machine learning model updated based at least in part on the first output data; determine a first protocol for generating first model update data to be sent to a second device, the protocol specifying a subset of model parameters, representing fewer than all model parameters, to include in a model update; determine, using the first protocol, a first subset of parameters of the second machine learning model; generate the first model update data using the first subset of parameters and one or more operations indicated by the first protocol; send, to the second device, the first model update data; receive, from the second device in response to the first model update data, second model update data; determine, by the first device using the second model update data, a second subset of parameters; and generate, by the first device using the second subset and the second machine learning model, a third machine learning model representing a version of the second machine learning model updated based at least in part on the second model update data.
14. The system of claim 13, wherein the at least one memory further comprises instructions that, when executed by the at least one processor, further cause the system to: determine a dataset for validating the second model update data; process the dataset using the second machine learning model to determine a first error rate; process the dataset using the third machine learning model to determine a second error rate; and determine, based at least in part on the first error rate and the second error rate, to process subsequent input data using the third machine learning model.
15. The system of claim 13, wherein the at least one memory further comprises instructions that, when executed by the at least one processor, further cause the system to: determine a dataset for validating the second model update data; process the dataset using the second machine learning model to determine second output data; process the dataset using the third machine learning model to determine third output data; and determine, based at least in part on the second output data and the third output data, to perform a partial update of the second machine learning model, wherein

generating the third machine learning model includes performing a weighted operation using the first subset of parameters and the second subset of parameters.

16. The system of claim 13, wherein the first machine learning model includes at least an input layer for receiving input data, an output layer for outputting the first output data, and at least one internal layer configured to receive first data from the input layer and send second data to the output layer, the at least one memory further comprises instructions that, when executed by the at least one processor, further cause the system to: determine, using the first protocol, to include a representation of the at least one internal layer in the first model update data; and determine the first subset of parameters based at least in part on first parameters corresponding to the at least one internal layer.

17. The system of claim 13, wherein the at least one memory further comprises instructions that, when executed by the at least one processor, further cause the system to: receive, from the second device in response to the first model update data, a second indication; and determine, based at least in part on the second indication, a second protocol for receiving the second model update data, wherein determining the second subset of parameters is further based at least in part on the second protocol.

18. The system of claim 13, wherein: the second machine learning model includes at least a first layer and a second layer; and the at least one memory further comprises instructions that, when executed by the at least one processor, further cause the system to generate the first model update data by reordering the at least first layer and second layer such that an order of the at least first layer and second layer in the first model update data is different from an order of the at least first layer and second layer in the first machine learning model.

19. The system of claim 13, wherein: the second machine learning model includes at least a first layer and a second layer, and the at least one memory further comprises instructions that, when executed by the at least one processor, further cause the system to determine the first subset by determining to include a representation of first parameters of the first layer in the first model update data, but not a representation of second parameters of the second layer.

20. The system of claim 13, wherein the at least one memory further comprises instructions that, when executed by the at least one processor, further cause the system to generate the first model update data by performing at least one mathematical function described by the first protocol on the parameters of the first subset; perform the at least one mathematical function includes generating one or more new parameter values; and the first subset includes the one or more new parameter values.